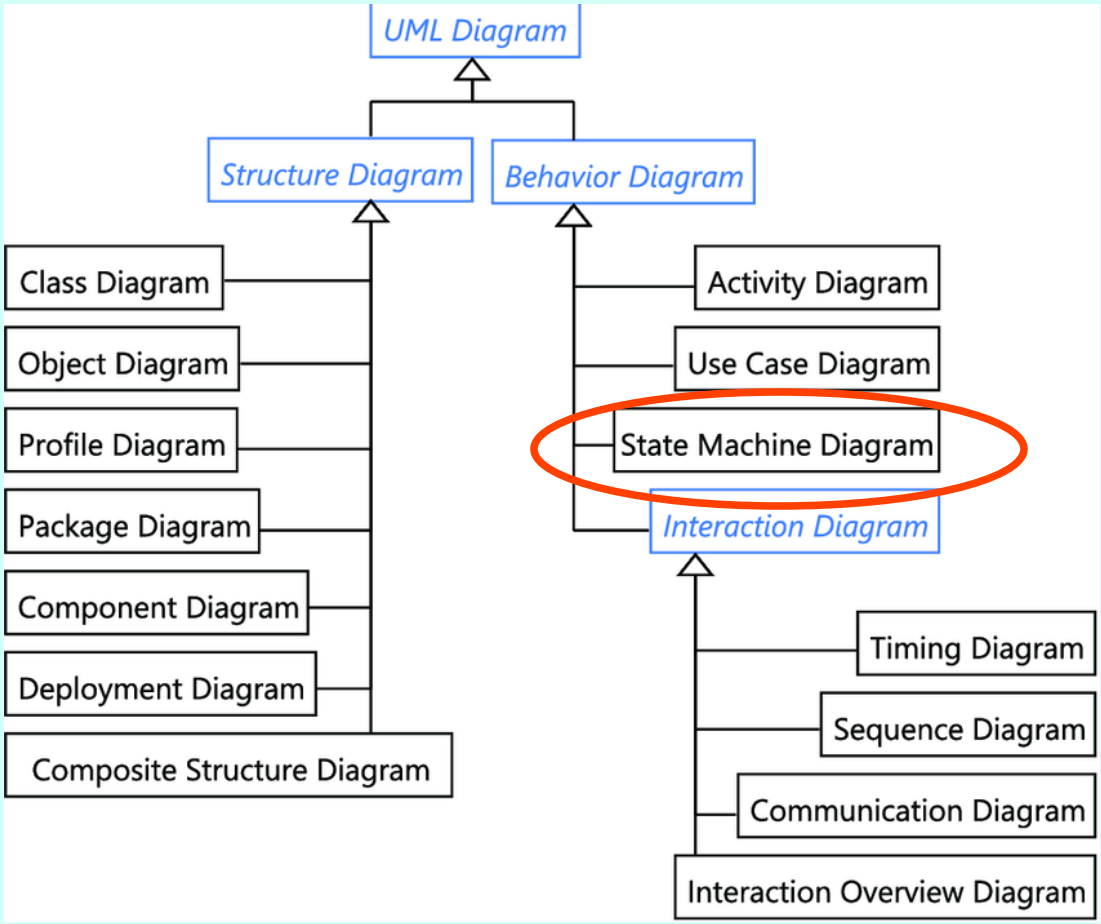
Introduction to the Unified Modeling Language (UML)

Part 2 – OO Design

Marcello Missiroli & Marcello Missiroli & Giancarlo Succi

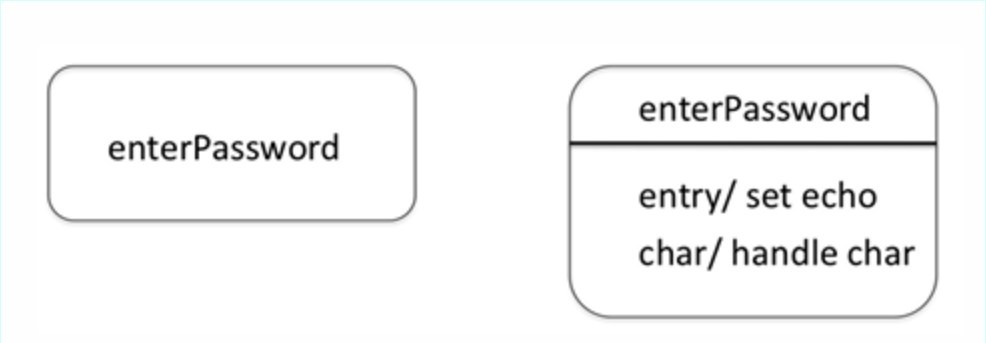
Statecharts

State diagram



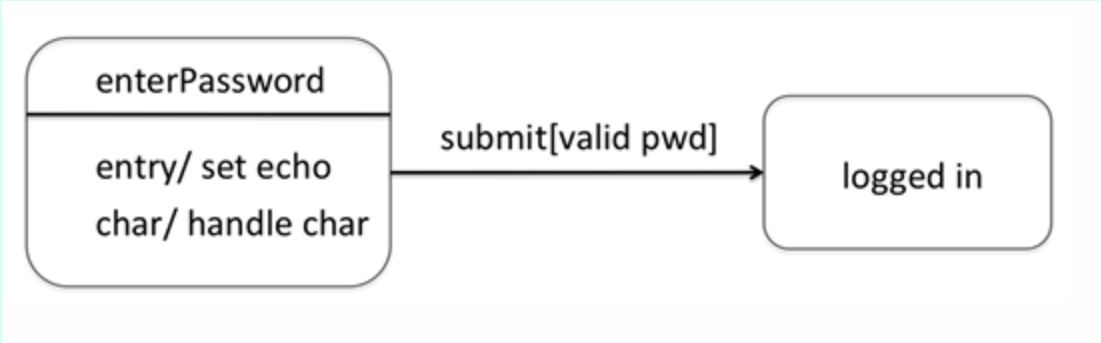
Statecharts (State Machine)

- Any UML classifier can be associated with a state machine that describes the behavior of its instances.
- A state is a condition or situation in the life of an object in which it:
  - satisfies a condition,
  - performs an activity, or
  - waits for an event.



Events & Transitions

- An **event** is "the specification of an occurrence that has a position in time and space."
- A **transition** is "the movement from one state to another in response to an event."



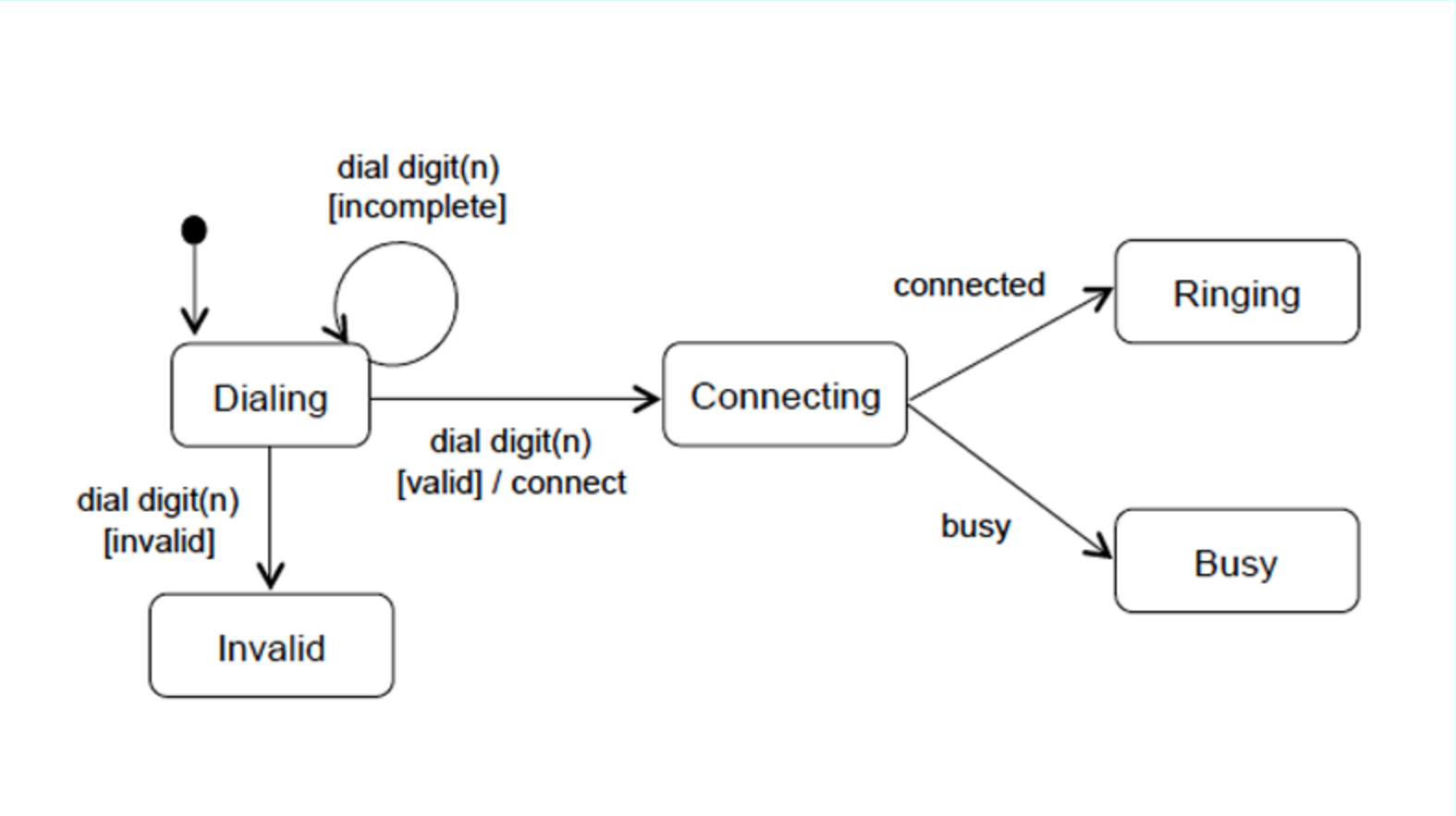
Semanthics

- The state machine receives event occurrences, which are stored in a queue and extracted one at a time.
- The semantics of these events follow a run-to-completion model:
  - an event occurrence is extracted only after the state machine has finished processing the previous one.
- If there are multiple executable transitions at a given time (e.g., two transitions from the same state triggered by the same event and both guard conditions are true), only one of them is executed.

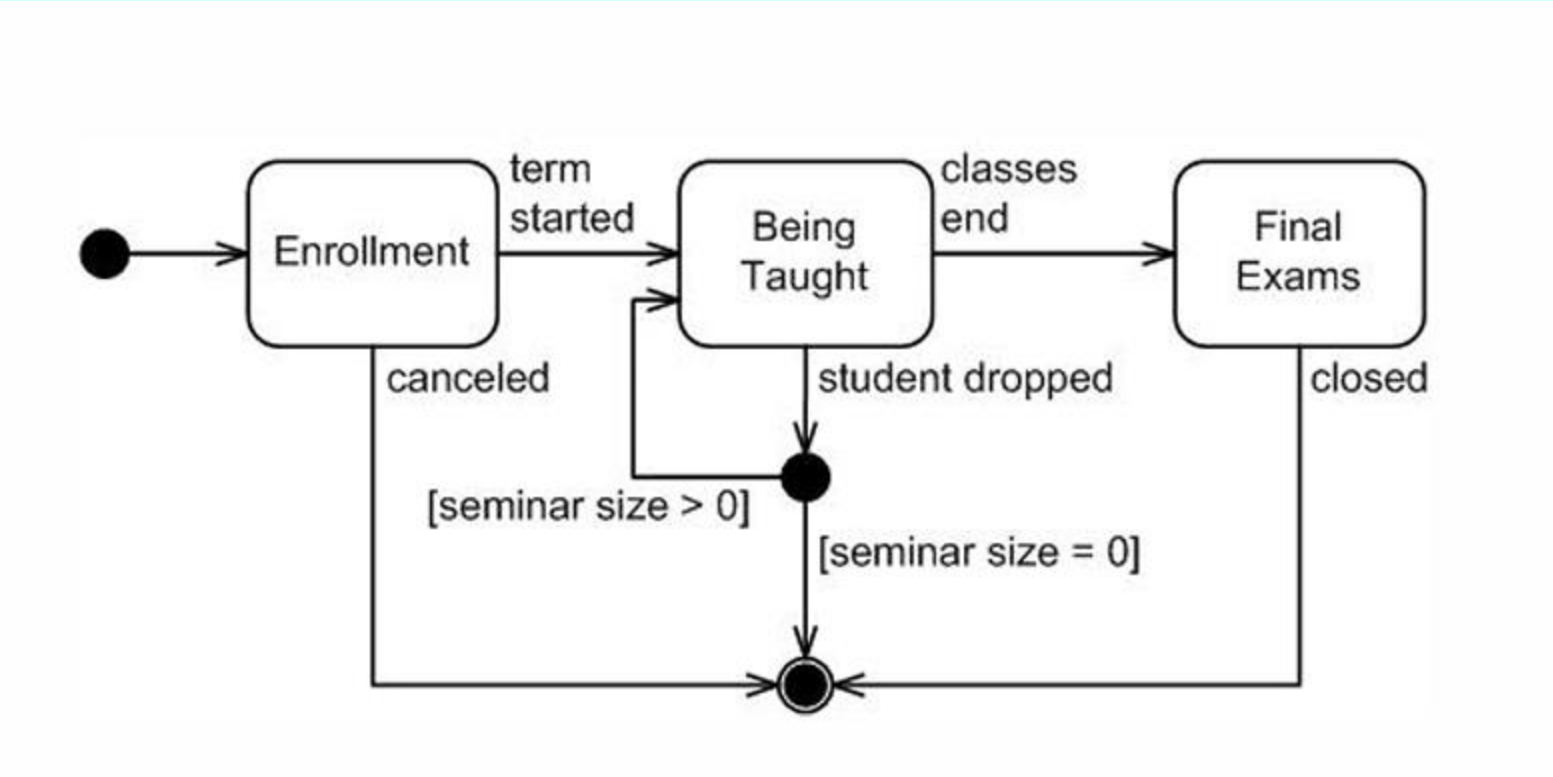
Transitions

- Each **transition**, in addition to the source and destination states, *can* specify:
  - Event: a trigger that activates the state change
  - Guard: a condition that, if true, allows the state change
  - Action: an operation that results from the state change
- Syntax: **`event [guard] / action`**
- The transition occurs in response to one of the events (when the guard is true), and at the moment of the transition, the context executes the specified action.
- All elements are *optional*.

Full example: phone call



Full example: university

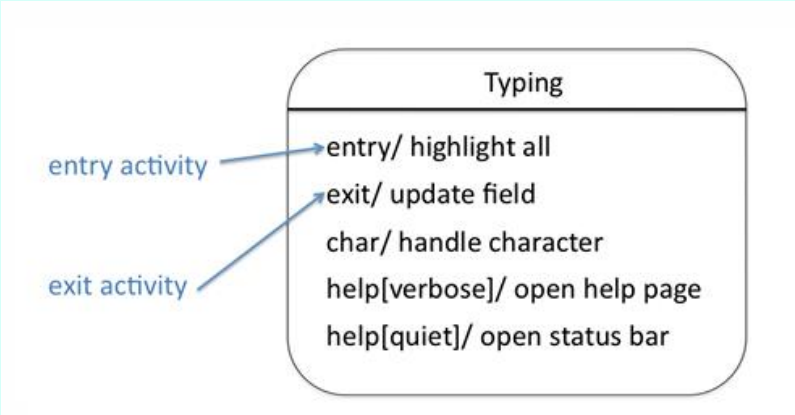


(pseudo) initial/final states

- The previous examples show two pseudo-states used to start and stop the state machine:
  - The black filled circle marks the start of execution. It is not a real state but a marker pointing to the initial state.
  - The black circle with a border (final node) indicates that execution has ended.
- These pseudo-states can appear in any number within a diagram (or within a composite state, which we will see later).

[Internal actions]

- A state can react to events (and evaluate conditions) without transitioning to a different state.
- These are called **internal activities**, shown in the second compartment of the state, and they follow the same syntax as transitions. They are similar to a self-transition, but the state does not exit and re-enter.
- Example: filling in a text field in a form.

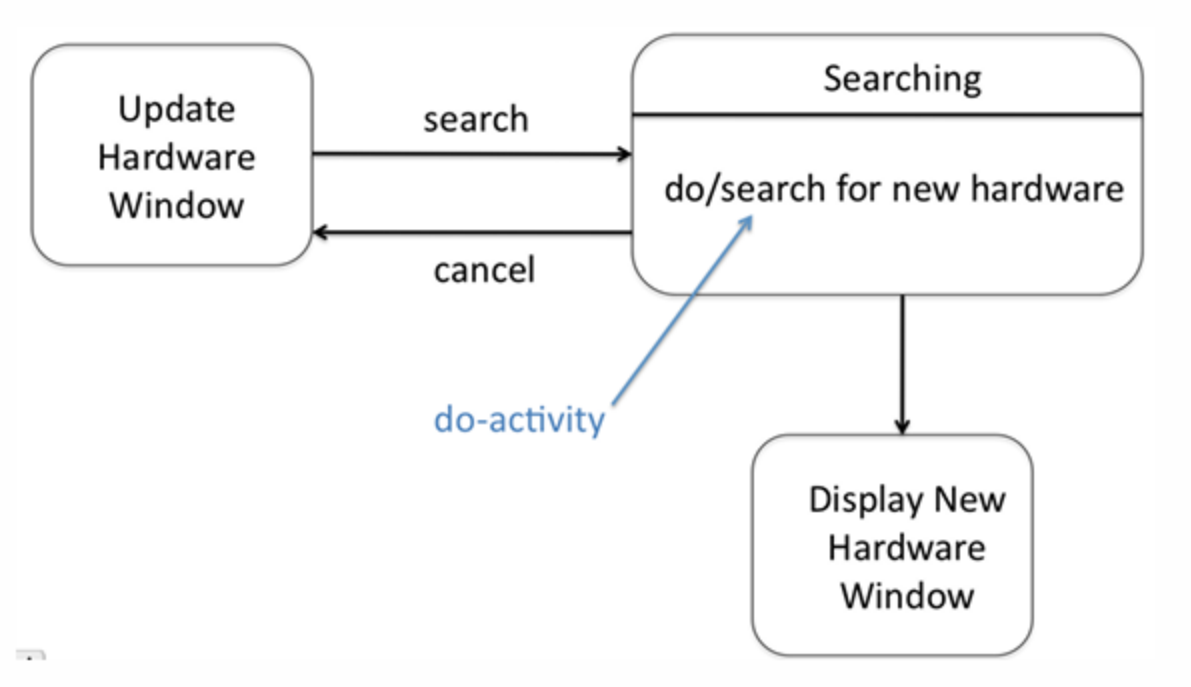


[Entry, exit, do]

- Within a state, certain special actions can be used, indicated by keywords:
  - **entry**: executed when the object enters the state
  - **exit**: executed when the object exits the state
  - **do** (do-activity): executed while the object is in the state
- A self-transition **always** triggers entry and exit actions, while internal activities do not.
- A do-activity is not instantaneous — it can last for a period of time and can be interrupted by other events.

[Entry, exit, do] - example

Search for a new device in an operating system

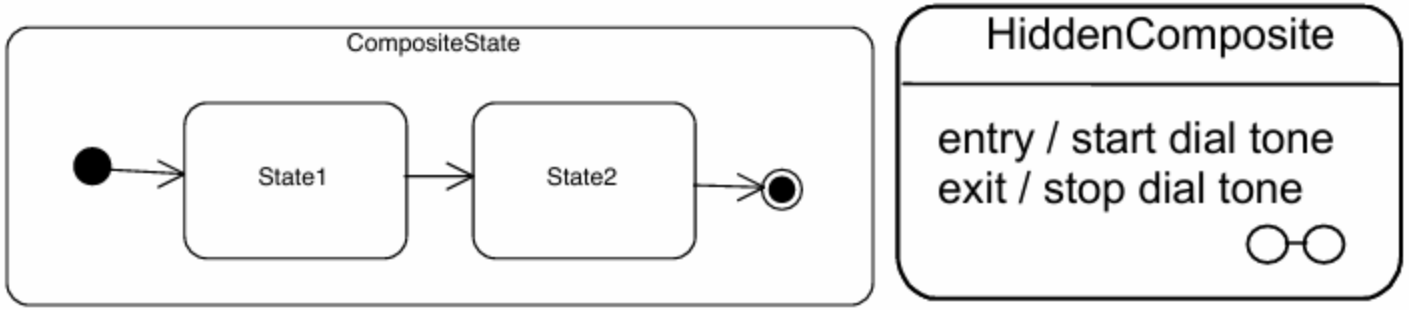


[Entry, exit, do]

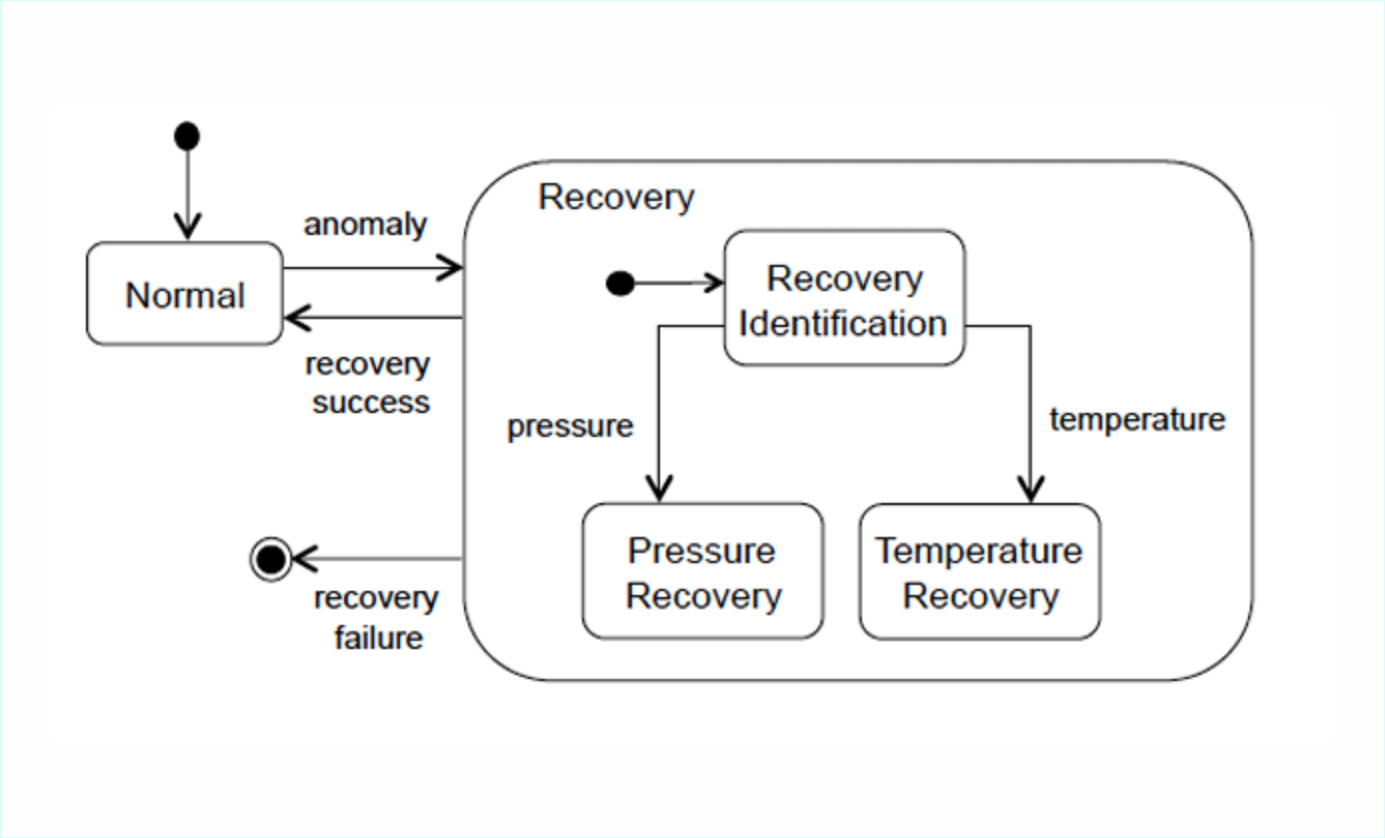
- Within a state, certain special actions can be used, indicated by keywords:
  - **entry**: executed when the object enters the state
  - **exit**: executed when the object exits the state
  - **do** (do-activity): executed while the object is in the state
- A self-transition **always** triggers entry and exit actions, while internal activities do not.
- A do-activity is not instantaneous — it can last for a period of time and can be interrupted by other events.

Composite states

- They allow for reducing the complexity of the model:
  - from the outside, a macro-state is visible, while internally it contains other states.
- It is also possible to create a state that refers to another state machine diagram (called a submachine state).
- An icon can be used to represent a composite state whose internal behavior is not shown.

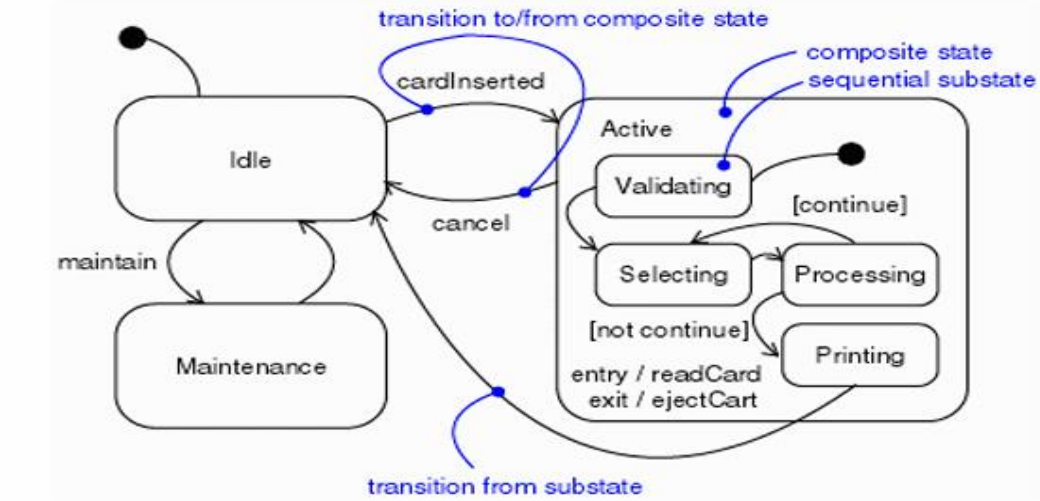


Composite states - example



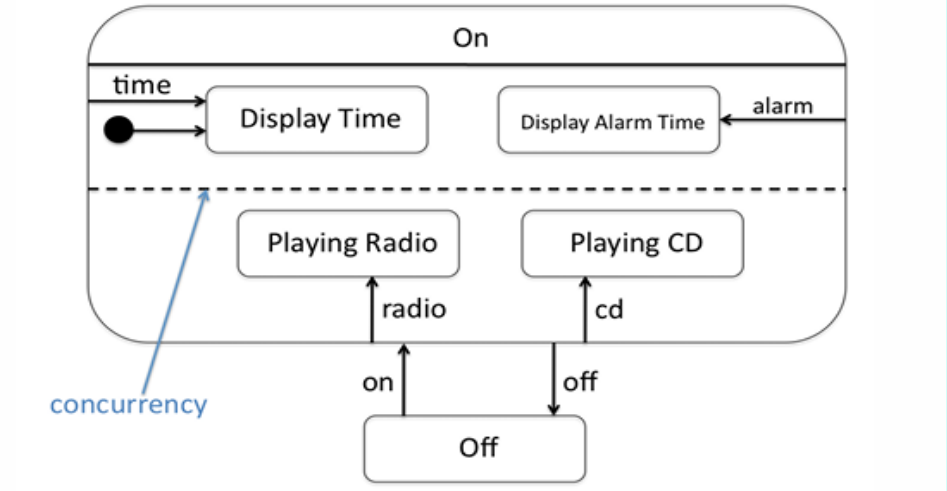
Composite states + transitions

- It is possible to define transitions from an internal state to external states.
- Outgoing transitions from the border of a composite state, triggered by an event, are inherited by all internal states.



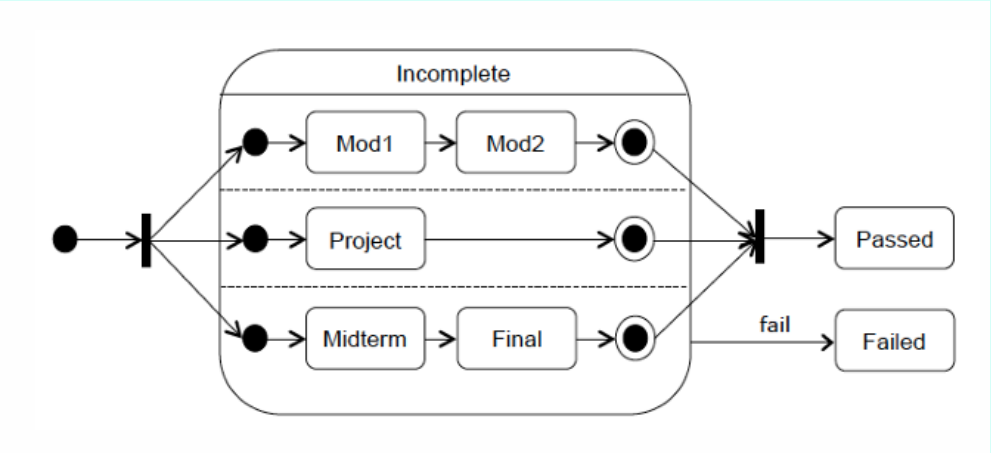
Composite states + concurrency

- Composite states are useful for modeling concurrency.
- The composite state is divided into **orthogonal** (sub-)**diagrams** that are executed in **mutual exclusion**.
- Without this approach, the diagram would be much less clear, as all possible combinations would need to be considered.
- Example: an alarm clock that either displays the time or plays music.

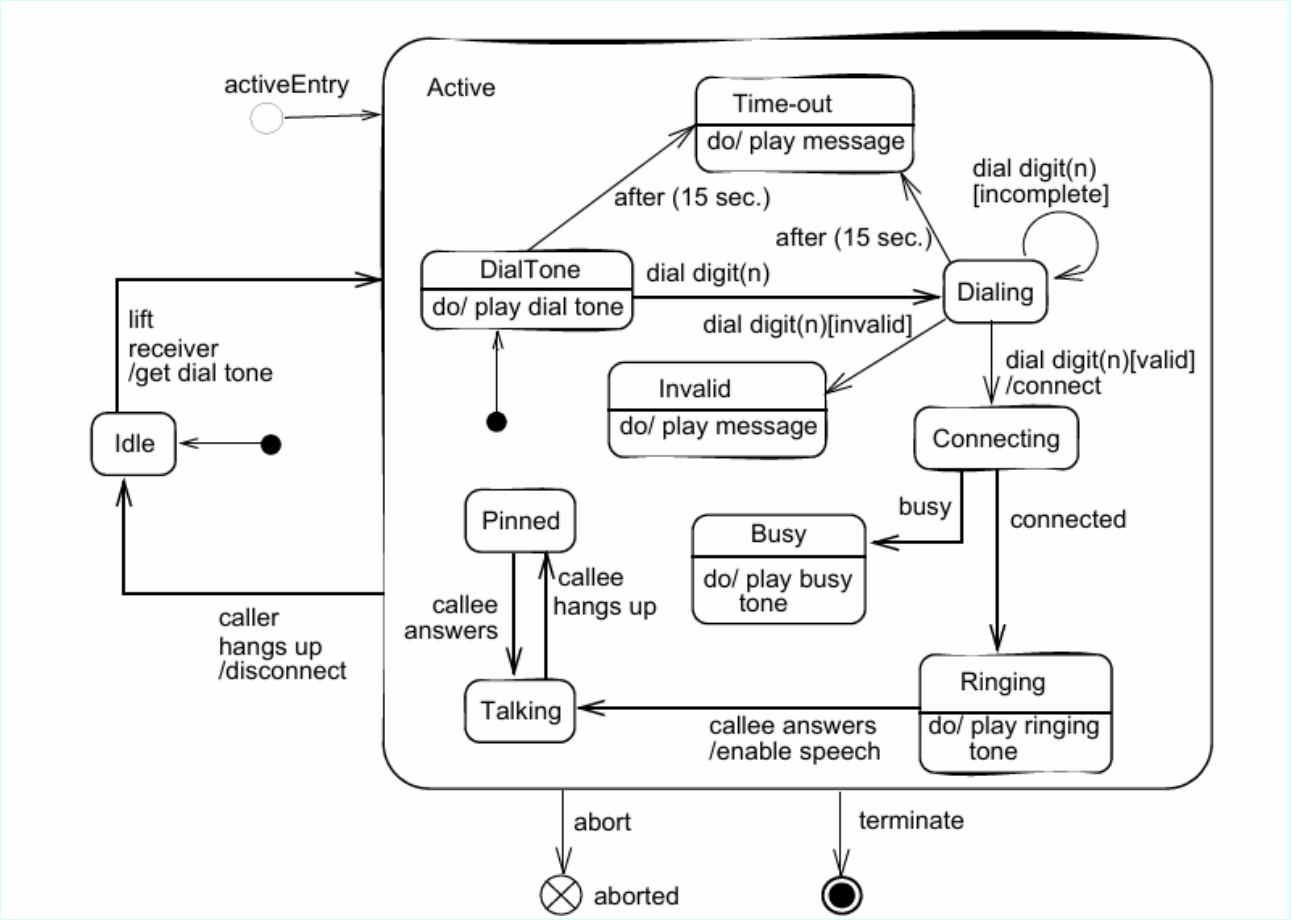


Composite states + synch

- Composite states are also useful for modeling **synchronization**.
- The composite state is divided into (sub-)diagrams, and fork and join operators (which we will see shortly) are used.
- Example: the assignments and exams a student must complete to finish a course.



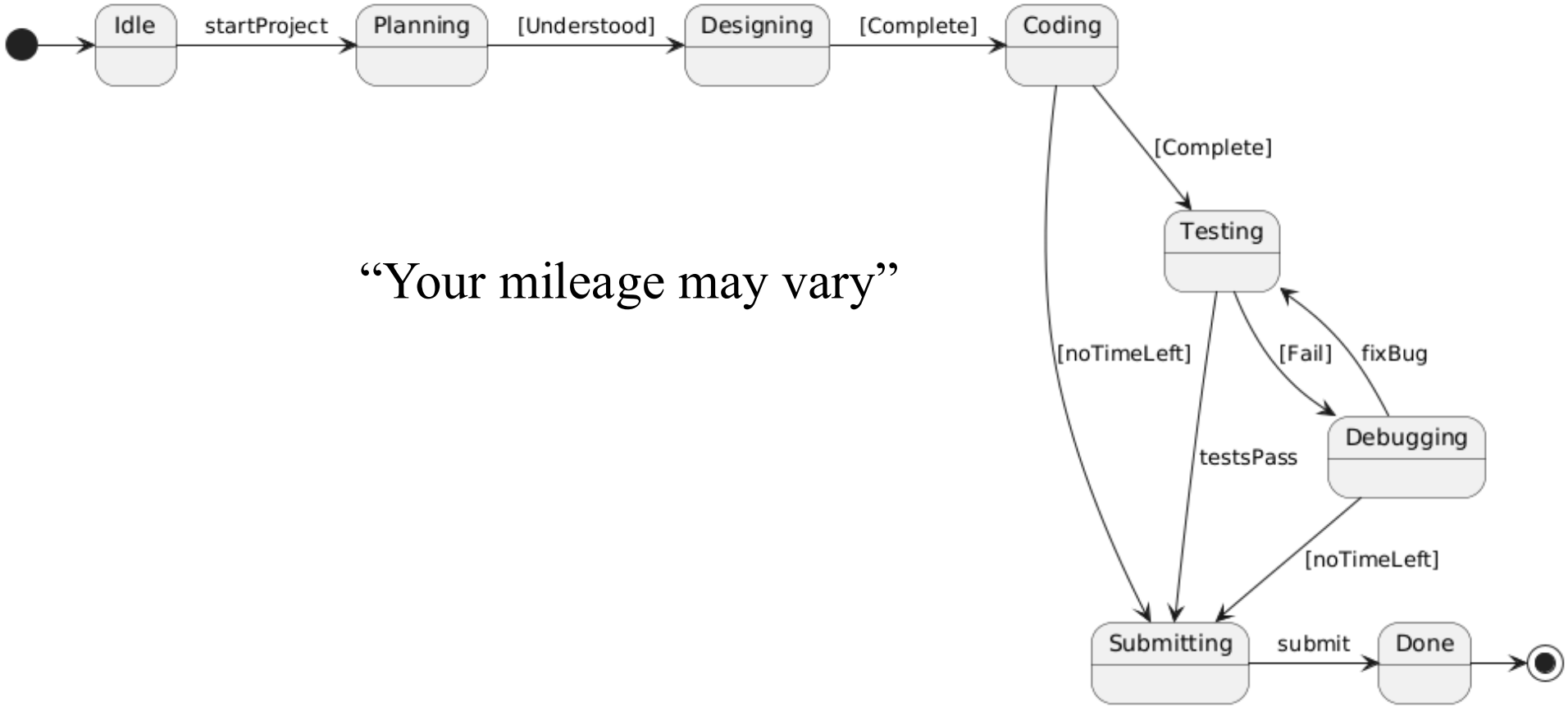
The Full Monty



Exercise 1

- Define a statechart diagram of your software development process

Exercise 1 Solution

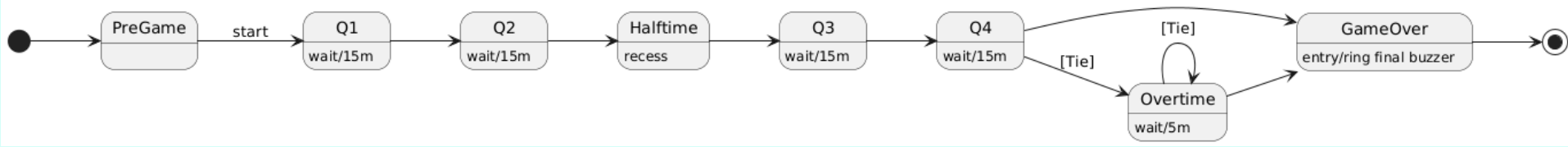


"Your mileage may vary"

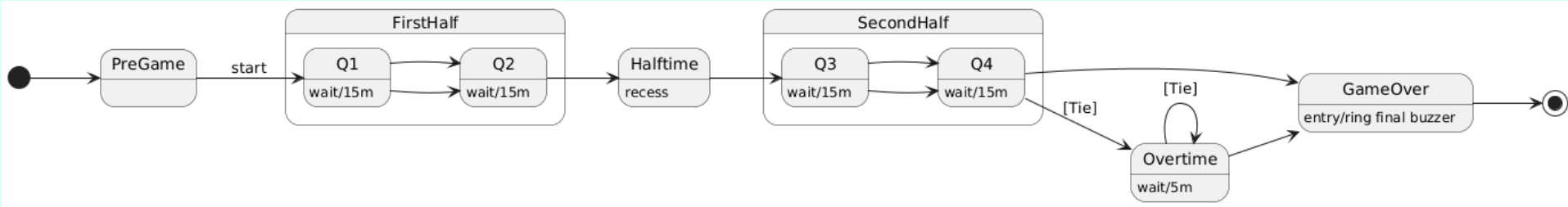
Exercise 2

- Define the statechart diagram of a basketball game

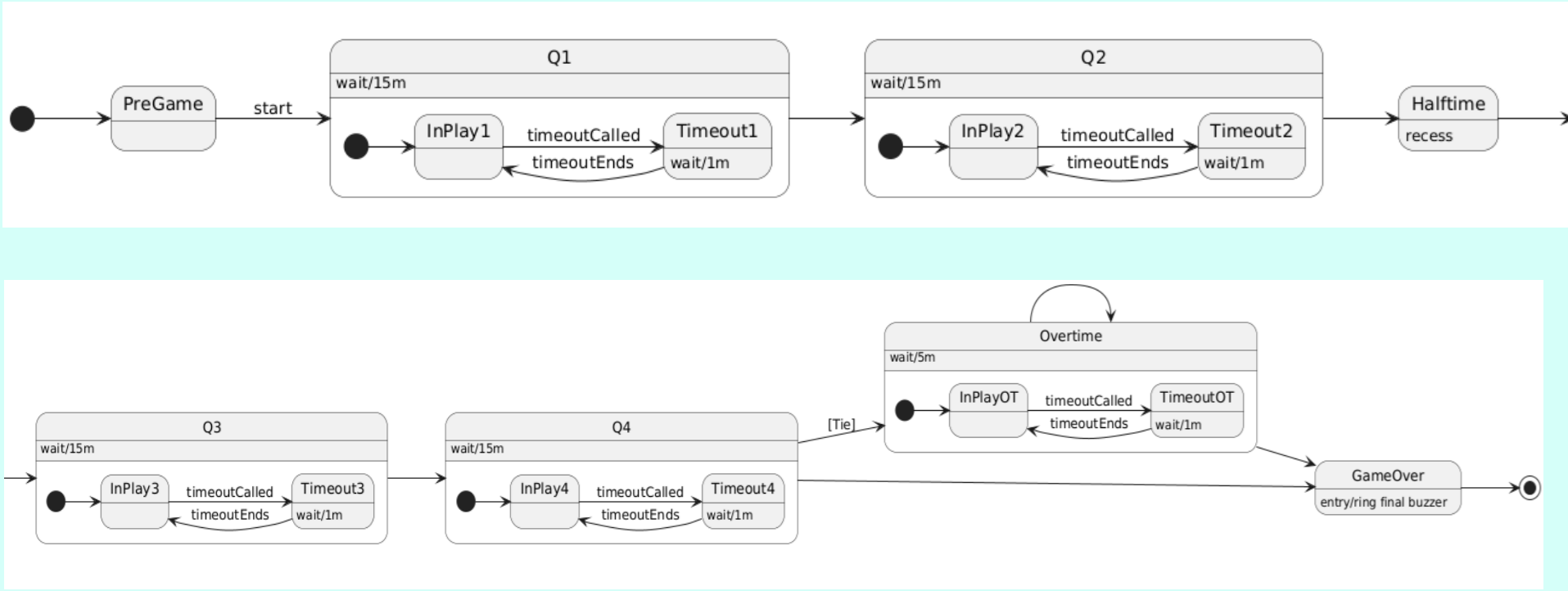
Exercise 2 simple solution



Exercise 2 solution w/substates



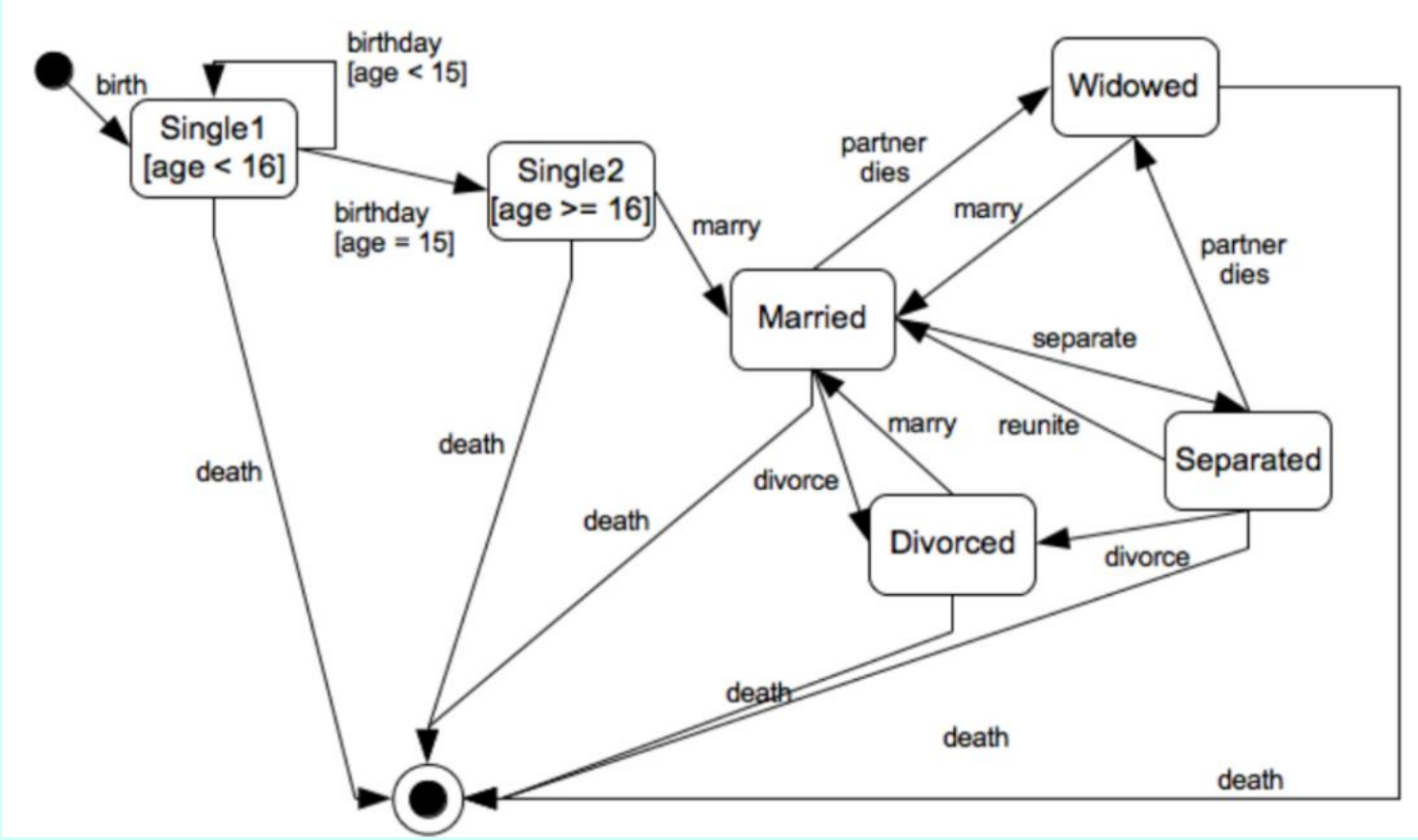
Exercise 2 solution w/timeouts



Exercise 3

- Create a state diagram that describes the civil (marital) status of a person (from birth to death).
- *Note: Under Italian civil law, marriage is allowed only for individuals who have reached the age of majority, that is, 18 years old.*
- As an exception, the juvenile court may grant emancipation to a minor aged at least 16, after verifying serious reasons and the individual's psycho-physical maturity, thereby allowing them to get married.

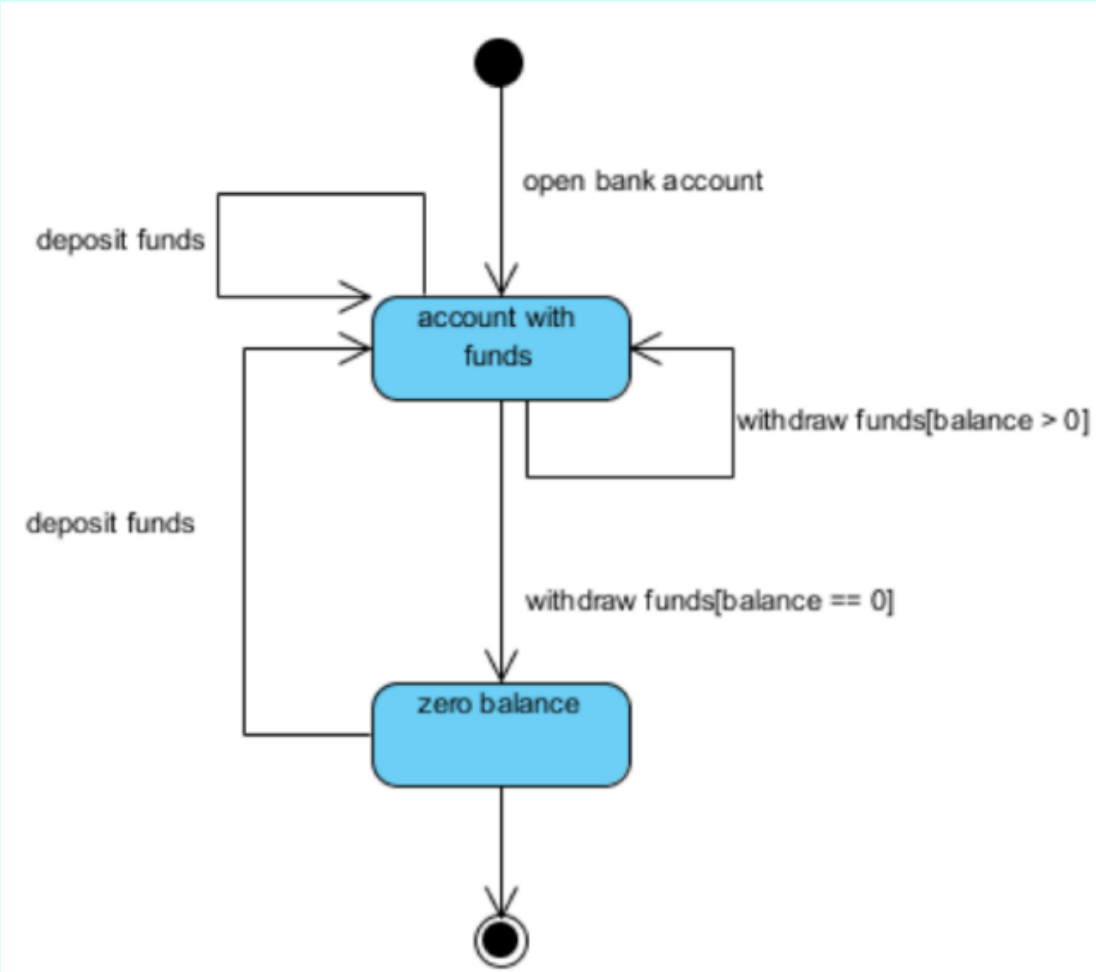
Exercise 3 - solution



Exercise 4

- A user performs the following operations on a bank account: open, withdraw, deposit.
- Draw a state diagram for the account to keep track of the balance.

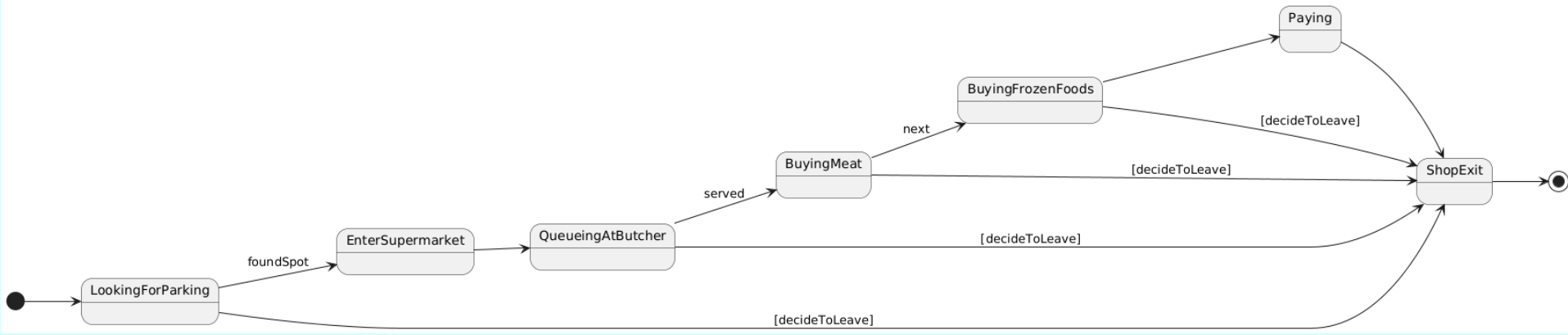
Exercise 4 - solution



Exercise 5

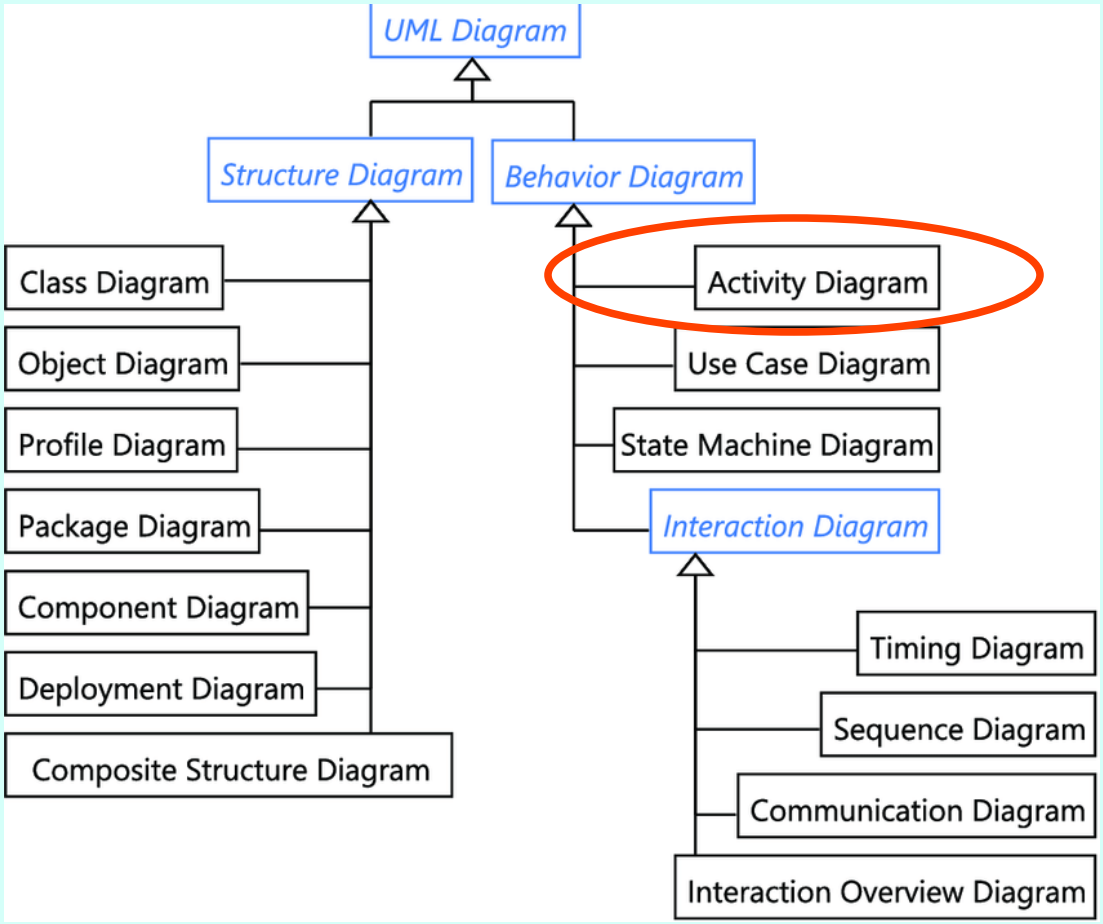
- Draw a state diagram (related to a customer) that describes a shopping experience in a supermarket:
- The customer looks for parking before starting the shopping trip.
- They must buy frozen foods and meat, and queue at the butcher counter before being served.
- After finishing the shopping, they pay at the register, but they may decide to leave at any time.

Exercise 5


Paying
BuyingFrozenFoods
[decideToLeave]
next
BuyingMeat
[decideToLeave]
ShopExit
served
EnterSupermarket
QueueingAtButcher
[decideToLeave]
foundSpot
LookingForParking
[decideToLeave]

Activity Diagrams

Activity diagrams



Activity diagrams

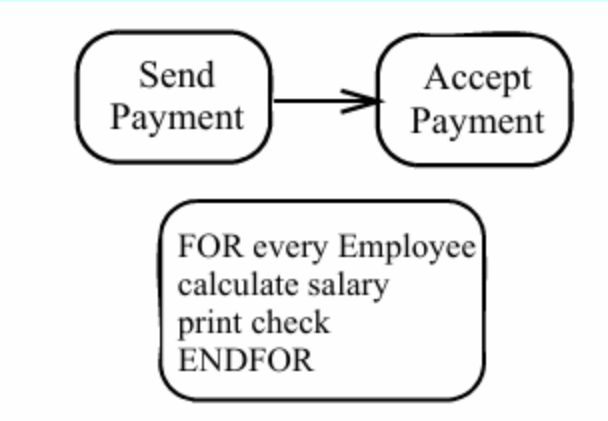
- ...model an activity related to any kind of object, for example:
  - Classes
  - Use cases
  - Interfaces
  - Components
  - Class operations
- Some uses of activity diagrams:
  - To model the flow of a use case (analysis)
  - To model the behavior of a class operation (design)
  - To model an algorithm (design)

Core components

- There are three kinds of nodes:
  - **Action** nodes:specify behavior unit.
  - **Object** nodes:specify object used as action input or output
  - **Control** nodes: specify activity flow

Action nodes

- An action can invoke an activity, a behavior, or an operation.
- *Like* other UML elements, actions support different levels of detail and different languages.
- *Unlike* messages in interaction diagrams, actions do not force the modeler to define all entities involved.
- Transitions (arrows) between actions can have a guard condition.

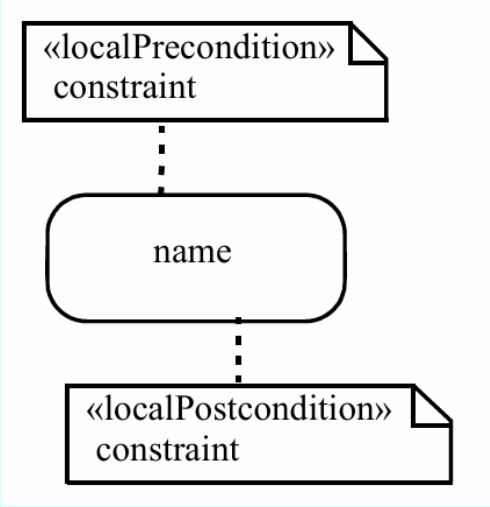


Token model

- To understand the semantics of activity diagrams, one must imagine entities called **tokens** that travel through the diagram.
- The flow of tokens defines the flow of the activity.
- Tokens can remain stationary at an action/object node while waiting for a condition to be satisfied on an outgoing arrow, or for a precondition/postcondition on the node itself.
- The movement of a token is atomic.
- An action node is executed when there are tokens on all incoming edges, and all preconditions are satisfied.
- At the end of the action, control tokens are generated on all outgoing edges.

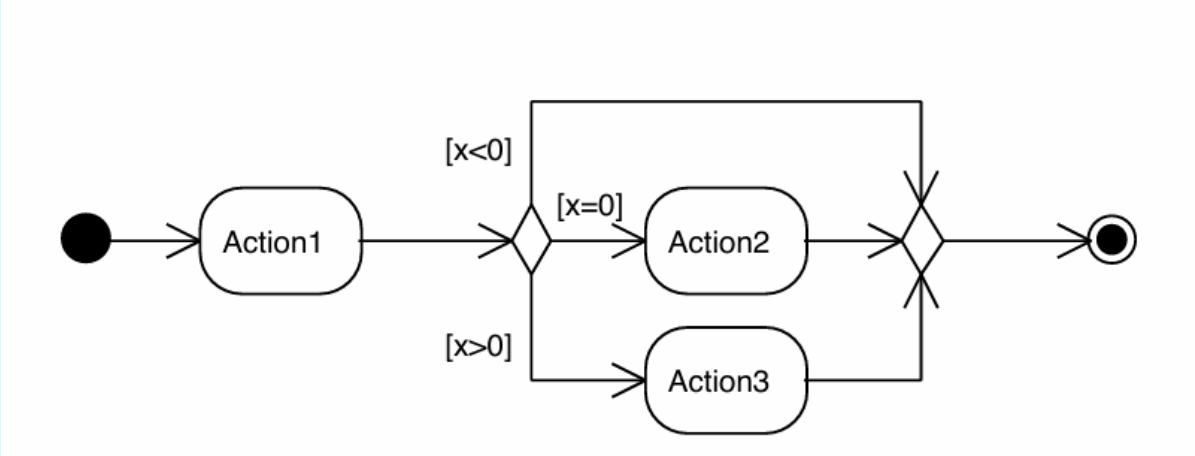
Preconditions/postconditions

- These are conditions, expressed in any form, that must be satisfied in order for an action to start or end
- (to allow a token to enter or exit).



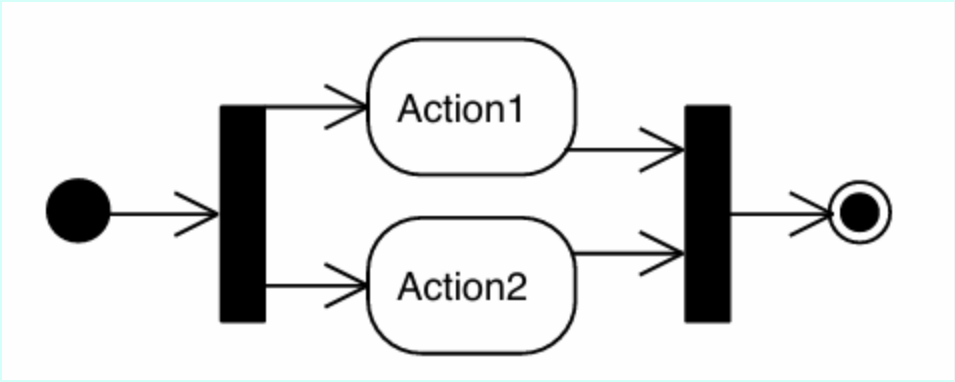
Decision & merge nodes

- **Decision** nodes have one input and multiple mutually exclusive outputs:
  - they copy the incoming token onto one of the output edges.
- **Merge** nodes have multiple inputs and a single output,onto which all incoming tokens are directed.



Fork/join nodes

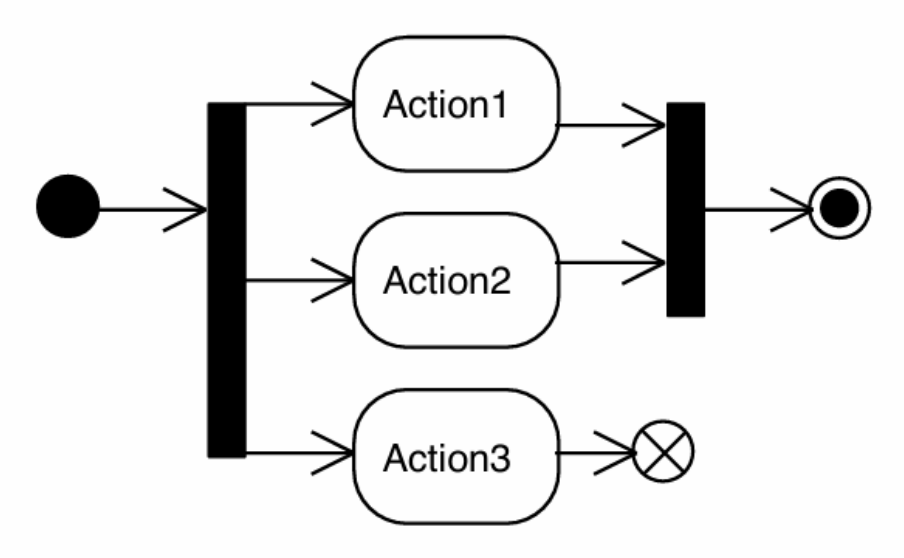
- **Fork** nodes have one input and multiple outputs: the incoming tokens are duplicated across all outputs.
- **Join** nodes have multiple inputs and a single output: when there are tokens on all inputs, at least one token is produced on the output.
- Fork nodes split an execution into **multiple concurrent flows**, where join nodes **synchronize and merge** those flows.



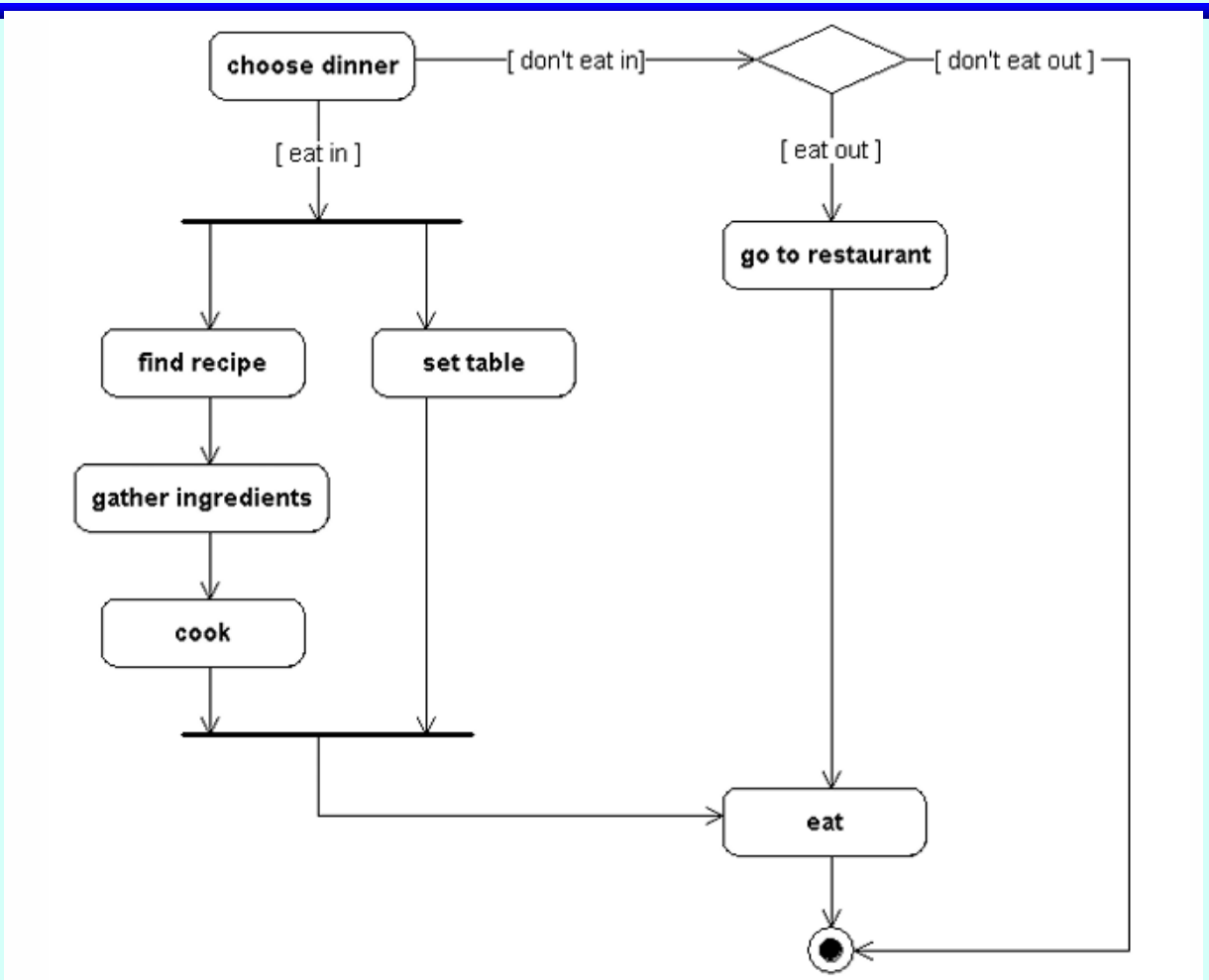
Final nodes

- When reached by a token, final nodes cause the termination **only of the flow** that reached them (i.e. kill thread)

However, reaching an **activity final node** causes the **termination of all flows** in the activity (i.e. kill process).



All at once

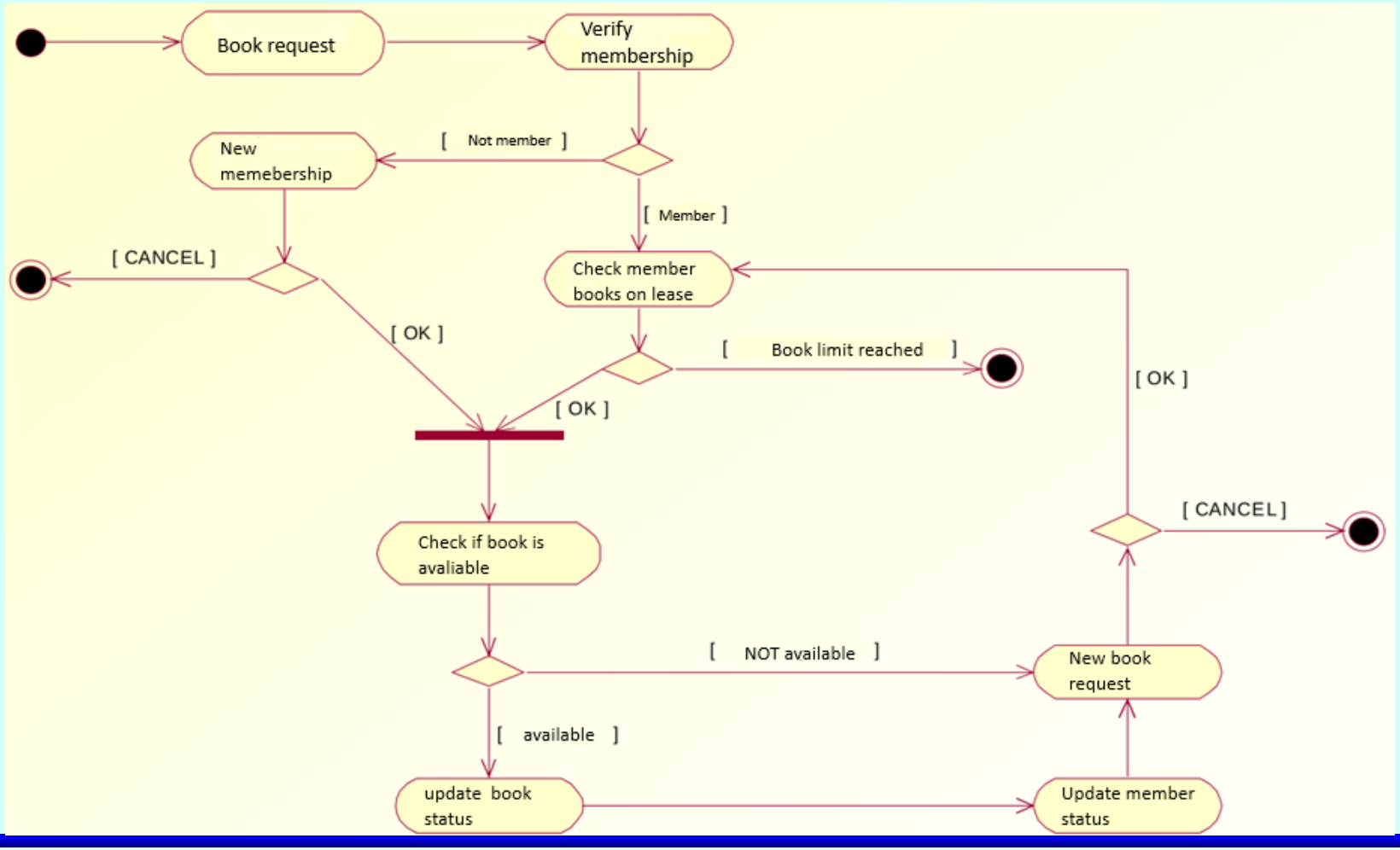

choose dinner
[don't eat in]
[don't eat out]
[eat in]
[eat out]
go to restaurant
find recipe
set table
gather ingredients
cook
eat

Exercise 1

Create an activity diagram that models the following domain:

- A library allowCreate an activity diagram that models the following domain:
- A library allows the borrowing of books and magazines.
- The system must keep track of ongoing loans and verify that the maximum number of items (6 volumes per person) has not been exceeded.
- Only registered members are allowed to borrow items.
- The availability of each item is constantly updated and must be checked at the time of the loan request.s the borrowing of books and magazines.
- The system must keep track of ongoing loans and verify that the maximum number of items (6 volumes per person) has not been exceeded.
- Only registered members are allowed to borrow items.
- The availability of each item is constantly updated and must be checked at the time of the loan request.

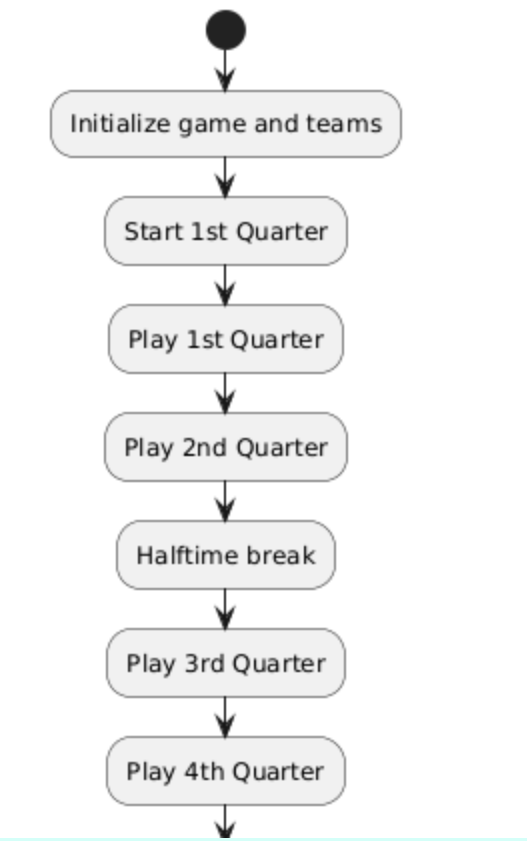
Exercise 1 – solution...sort of

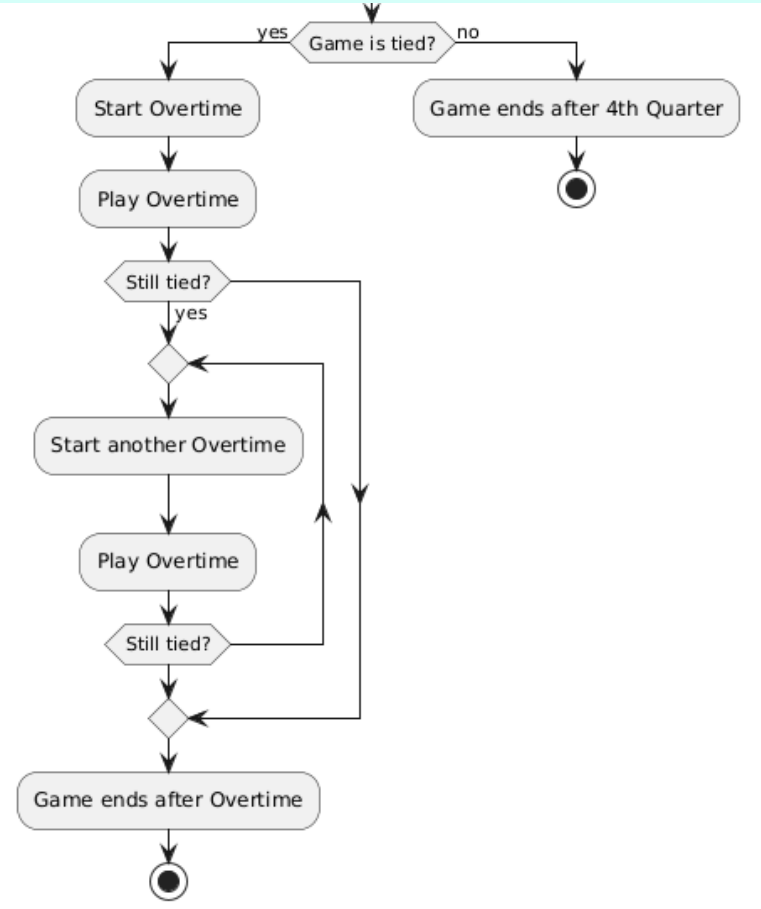


Exercise 2

- Define an activity diagram for the previously defined basketball game
- *Warning: loops are ugly*

Exercise 2


Initialize game and teams
Start 1st Quarter
Play 1st Quarter
Play 2nd Quarter
Halftime break
Play 3rd Quarter
Play 4th Quarter

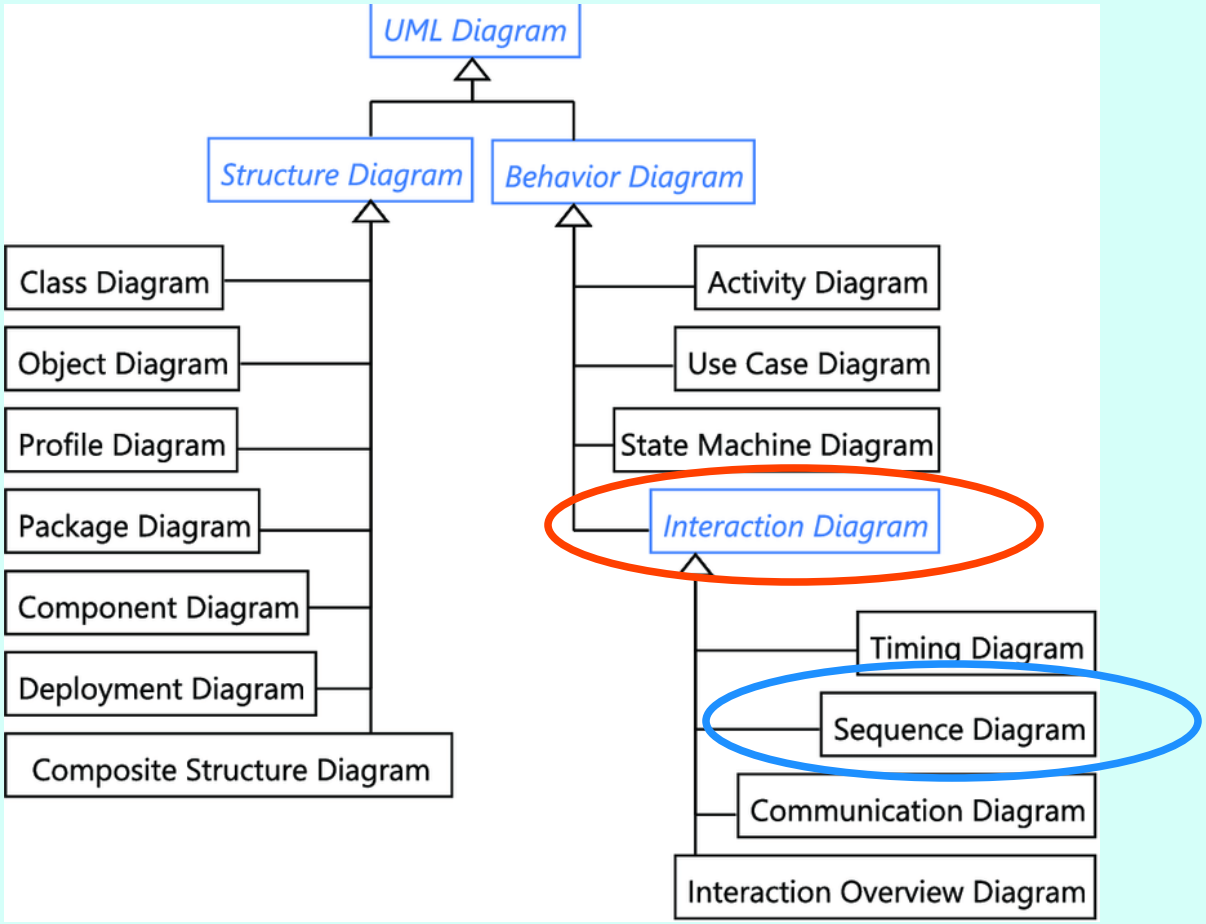

Game is tied?
yes
no
Start Overtime
Game ends after 4th Quarter
Play Overtime
Still tied?
yes
Start another Overtime
Play Overtime
Still tied?
Game ends after Overtime

Proposed Exercise

Define an activity diagram for your software development process

Interaction diagrams

State diagram



Interaction diagrams

- An interaction diagram requires two elements:
  - A behavior to be realized, derived from a classifier (the context classifier), for example:
    - An use case
    - a class operation
  - A set of elements that realize the behavior, for example:
    - actors
    - class instances
- These ingredients are typically derived from diagrams created earlier in the modeling process.

Lifelines

In interaction diagrams, classifiers such as classes generally do not appear directly. Instead, one finds:

- **Instances of classifiers** (e.g., objects, instances of actors, etc.)
- **Classifier lifelines**, which represent roles rather than specific objects

The distinction between instances and lifelines is subtle, but in general it is preferable to use lifelines.

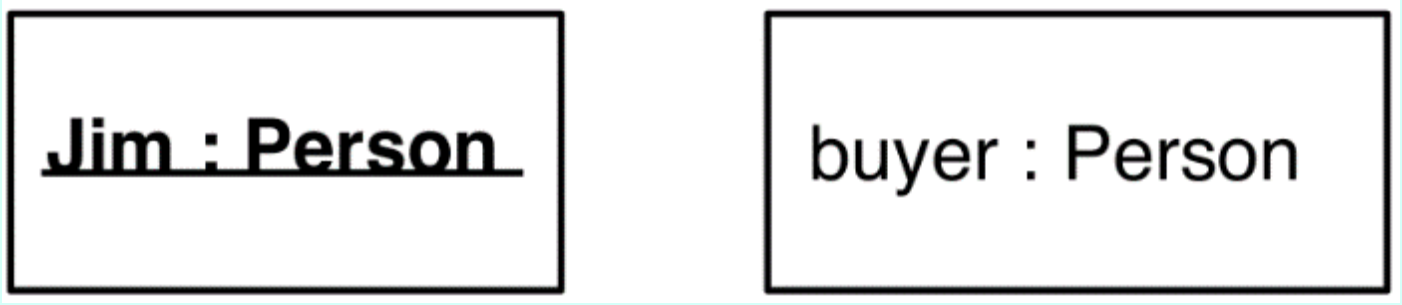
These diagrams also include — indeed, are primarily composed of — messages, namely the signals exchanged among classifier instances and/or lifelines.

Lifelines

An instance represents one specific object, and only that object.

A lifeline is more general. It represents an arbitrary instance of a classifiers, provides mechanisms to specify how that instance is selected and expresses the role played by an instance without concern for its concrete identity.

Lifelines



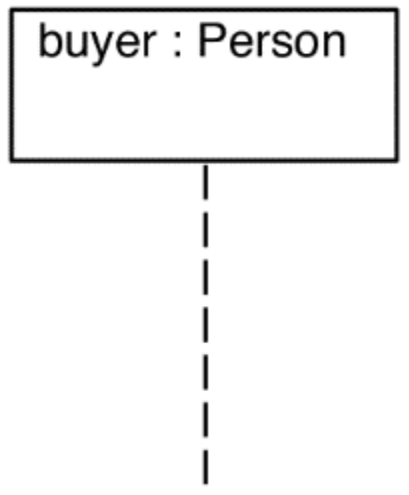
Both instances and lifelines use the notation of their classifier, but lifelines are not underlined.

The full notation for a lifeline is:
`name [selector] : type`

The selector is an optional Boolean expression that allows one to choose a particular representative of the classifier (e.g., [id = "1234"]). Without a selector, the lifeline denotes an arbitrary instance of the classifier.

Sequence diagrams

- They highlight the temporal order of method invocations.
- A vertical dashed line represents a timeline.
- Events occur in their order along the line; the distances between them are irrelevant.
- Multiple lifelines interact to realize the offered behavior by exchanging messages.

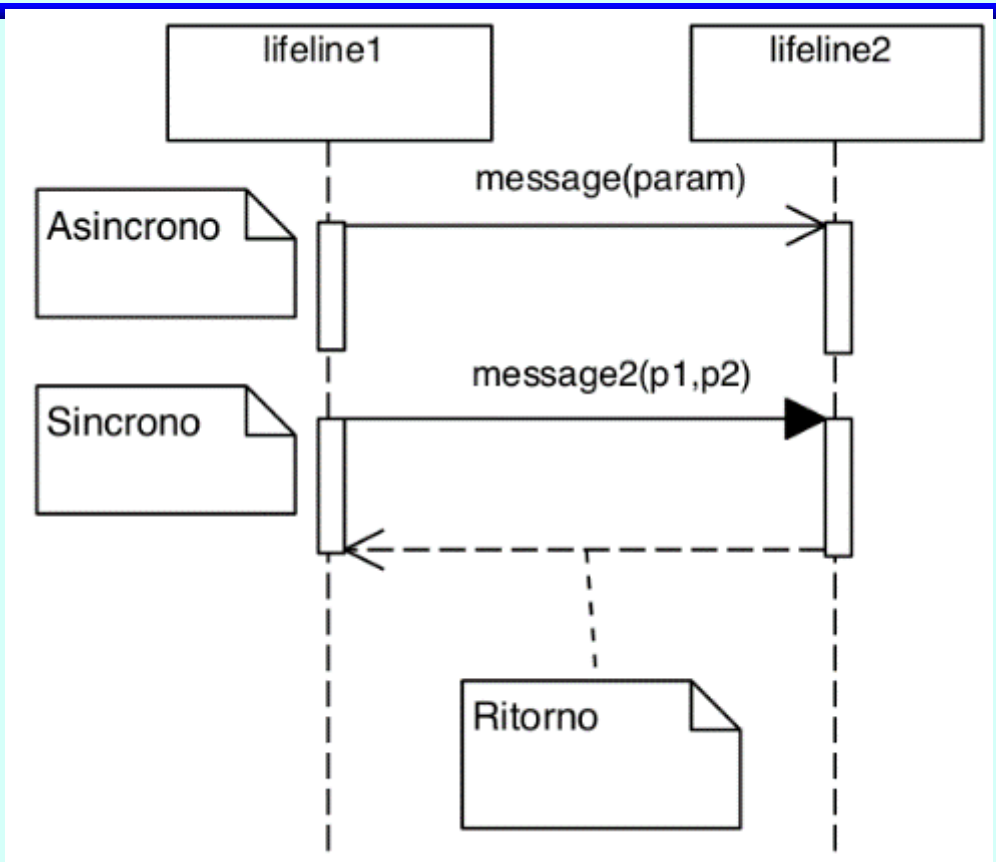


Sequence diagrams

They represent communications between lifelines: signals, operation calls, object creation, and object destruction.

Seven message types are defined:

- synchronous call
- asynchronous call
- return from call
- creation
- destruction
- found message
- lost message



Synchronous vs. asynchronous communication. Activation rectangles (indicating a focus of control) are optional.

Synch vs. Asynch

Synchronous messages block the sender until the recipient's return message is received.

In asynchronous messages, the sender does not expect a return message and proceeds immediately.

Return messages are optional and are generally included only when they do not hinder the readability of the diagram or to indicate return values.

The distinction between synchronous and asynchronous messages usually emerges during the design phase.

During the analysis phase, it is advisable to indicate all messages as synchronous (this is the most restrictive case).

Message syntax

In the analysis phase, it is generally sufficient to include just the name of the message.

If more detail is desired, a list of parameters *can* be included in parentheses.

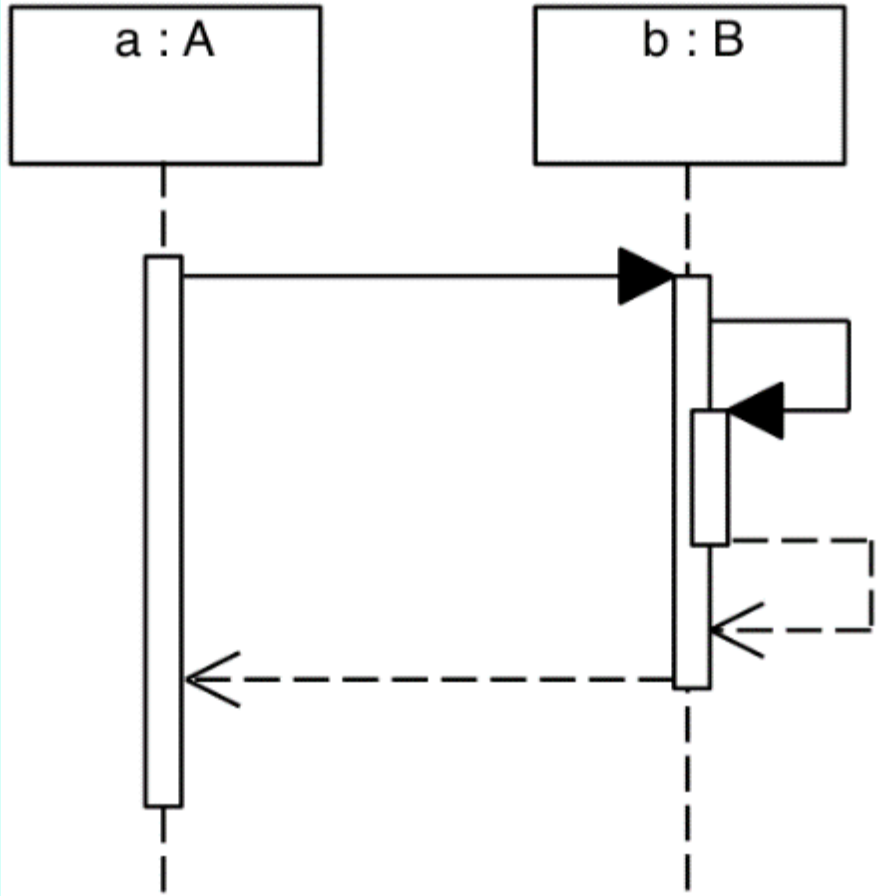
Guards can also be used to check conditions.

It is possible to indicate formal parameters, actual parameters, or both (in this order: **`getArea(shape=g)`**).

Nested activation

Lifelines can send messages to themselves.

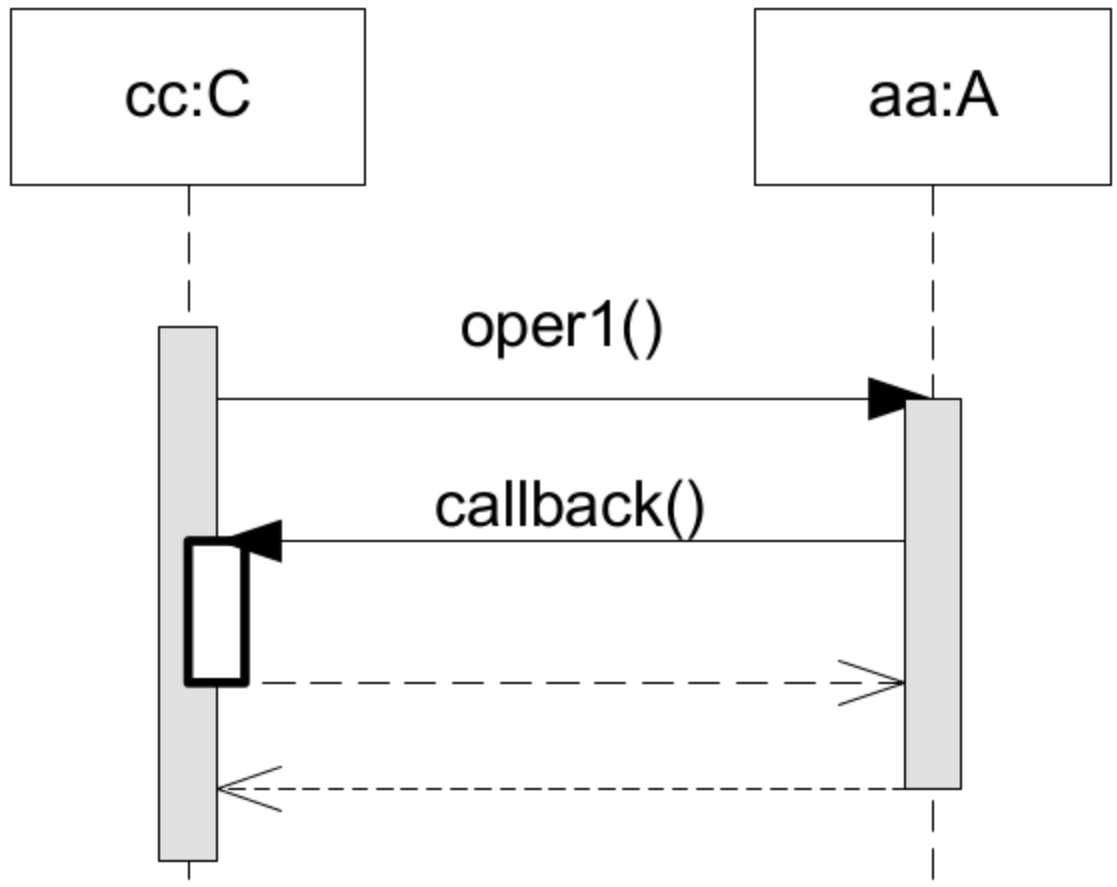
This is a common case (e.g., invocation of private operations).



Nested activation

Lifelines can send messages to themselves.

This is a common case (e.g., invocation of private operations).

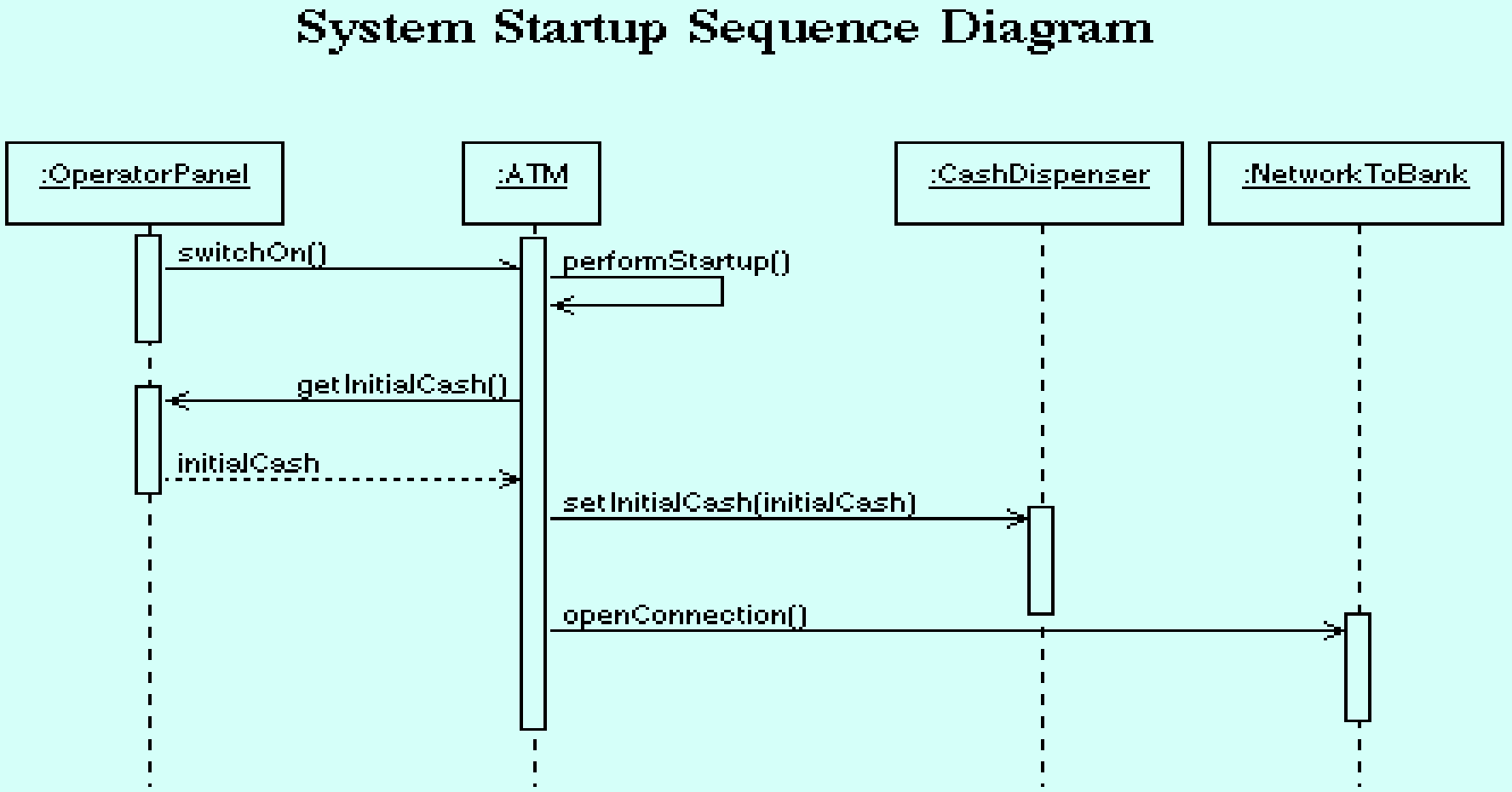


Exercise 1

Create a sequence diagram to model the following domain:

The System is activated when the operator sets the switch to the “on” state. The operator is then asked to enter the amount of cash available in the ATM, and afterwards a connection with the bank is established. At this point, customers can be served.

Exercise 1- solution



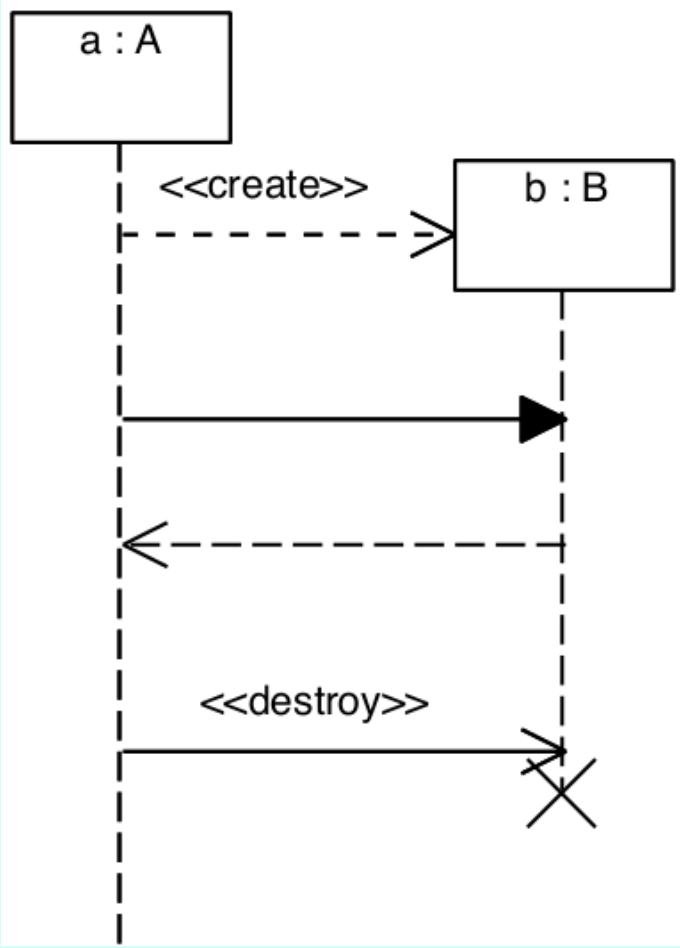
Msg creation/destruction

In a creation message, the stereotype can be followed by the name of an operation and its parameters (constructor).

This becomes necessary when working with programming languages that do not have explicit constructors.

Different object-oriented programming languages handle destruction in different ways (explicit, garbage collector).

In the analysis phase, it is sufficient to assume that after destruction the object is no longer accessible.



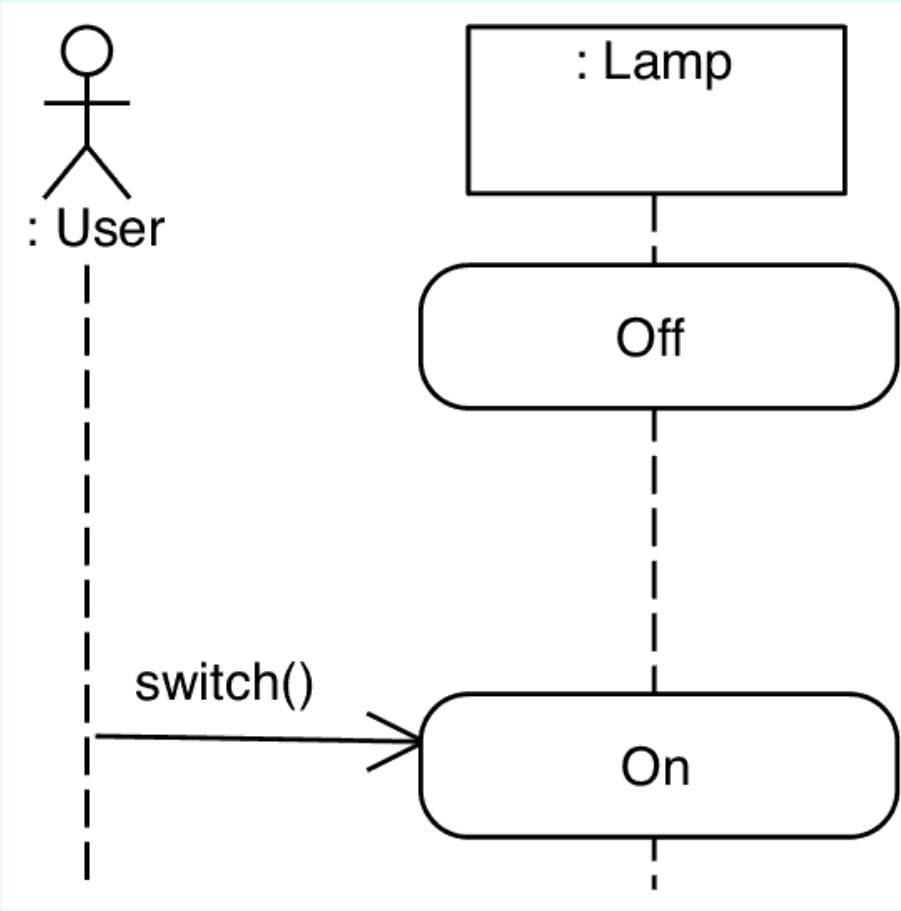
States

It is possible to indicate the state of an object, representing it with rounded rectangles.

Generally, only the main states of the object are shown, highlighting the messages that cause a state change.

State changes are generally the consequence of one or more messages.

Note: state diagrams are more suitable for representing these situations.



Fragments

A combined fragment is a sub-area of a sequence diagram that encloses part of the interaction and the way it is executed. The elements of a combined fragment are:

- an **operator**, which specifies how the fragment is executed
- one or more **operands**, sub-areas of the sequence diagram that may be executed
- zero or more **guard** conditions, expressions that determine which operands are executed.

The UML syntax:

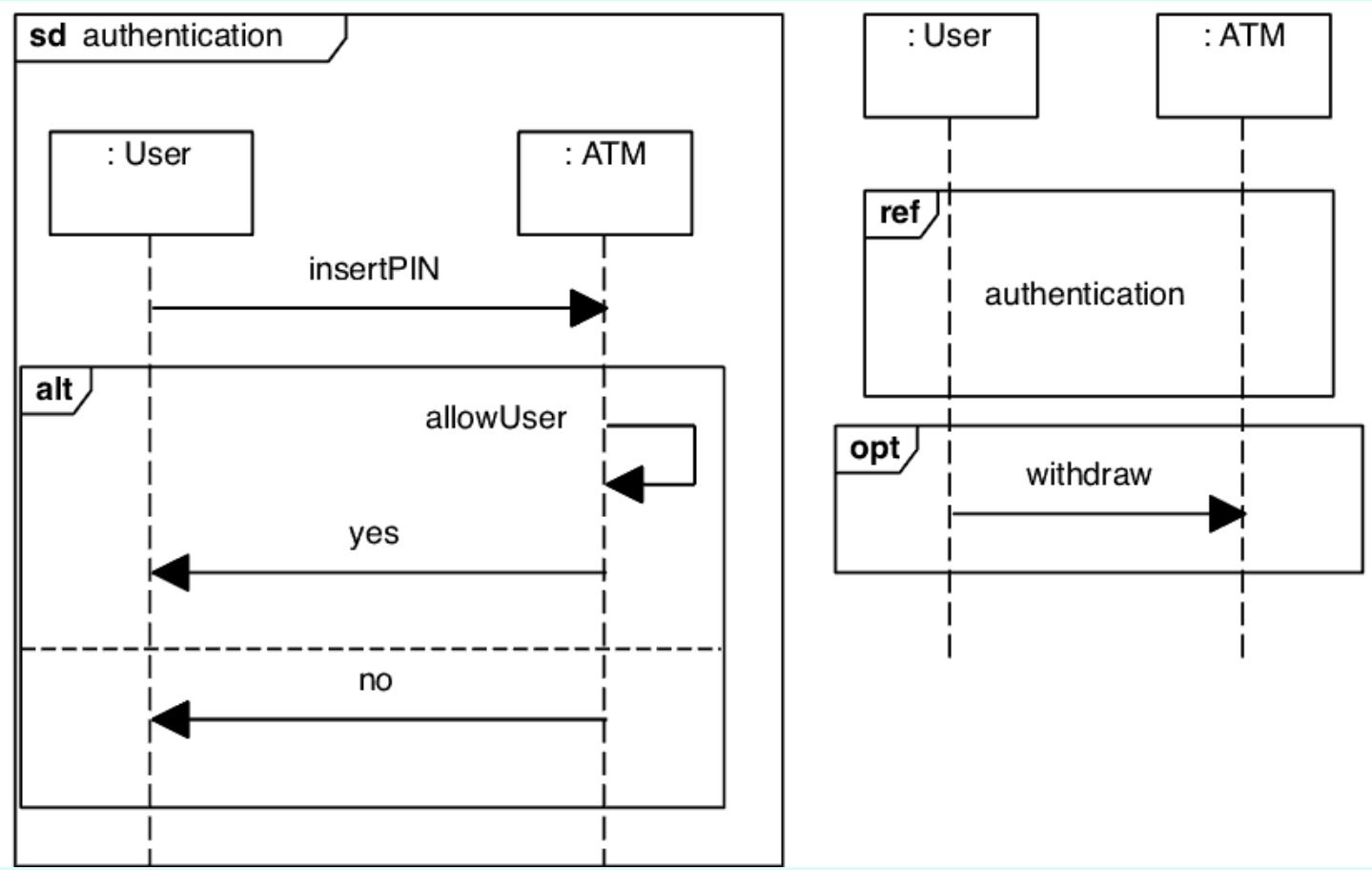
- a **rectangle** with the name of the **operator** in the top-left corner, enclosed in the diagram symbol (a rectangle with a cut corner); the operands follow as horizontal strips in sequence, separated by dashed lines
- the guard, if present, appears **after the operator or at the top of an operand** inside square brackets
- the rectangle of the combined fragment extends horizontally to include the lifelines involved in the interaction
- the guard may include any type of boolean condition

Operators

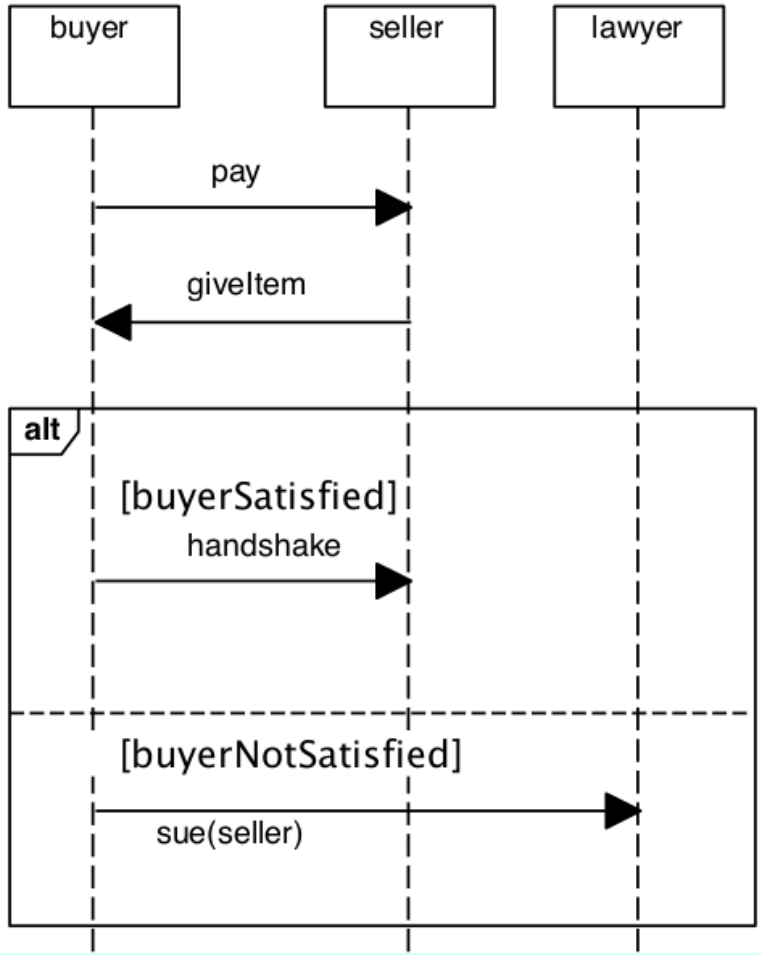
UML specifies many types of operators, but only some are frequently used (those that correspond to flow-control primitives defined in programming languages). The most commonly used are:

- **opt**: accepts only one operand, which is executed if and only if the guard evaluates to true.
- **alt**: accepts any number of operands and executes at most one of them.
- **loop**: a single operand, special syntax: loop min,max [condition]; executes the operand min times, and then at most (max – min) additional times while the condition is true. min and max can be omitted (the condition is very flexible and may be expressed in natural language).
- **break**: corresponds to break.
- **gate**: any point on the outer frame of the diagram, which can be the source or the destination of a message arrow
- **ref**: a reference to another interaction (another sequence diagram). It allows you to modularize diagrams by replacing a portion of the interaction with a call to a separate diagram, which can be reused in different contexts.

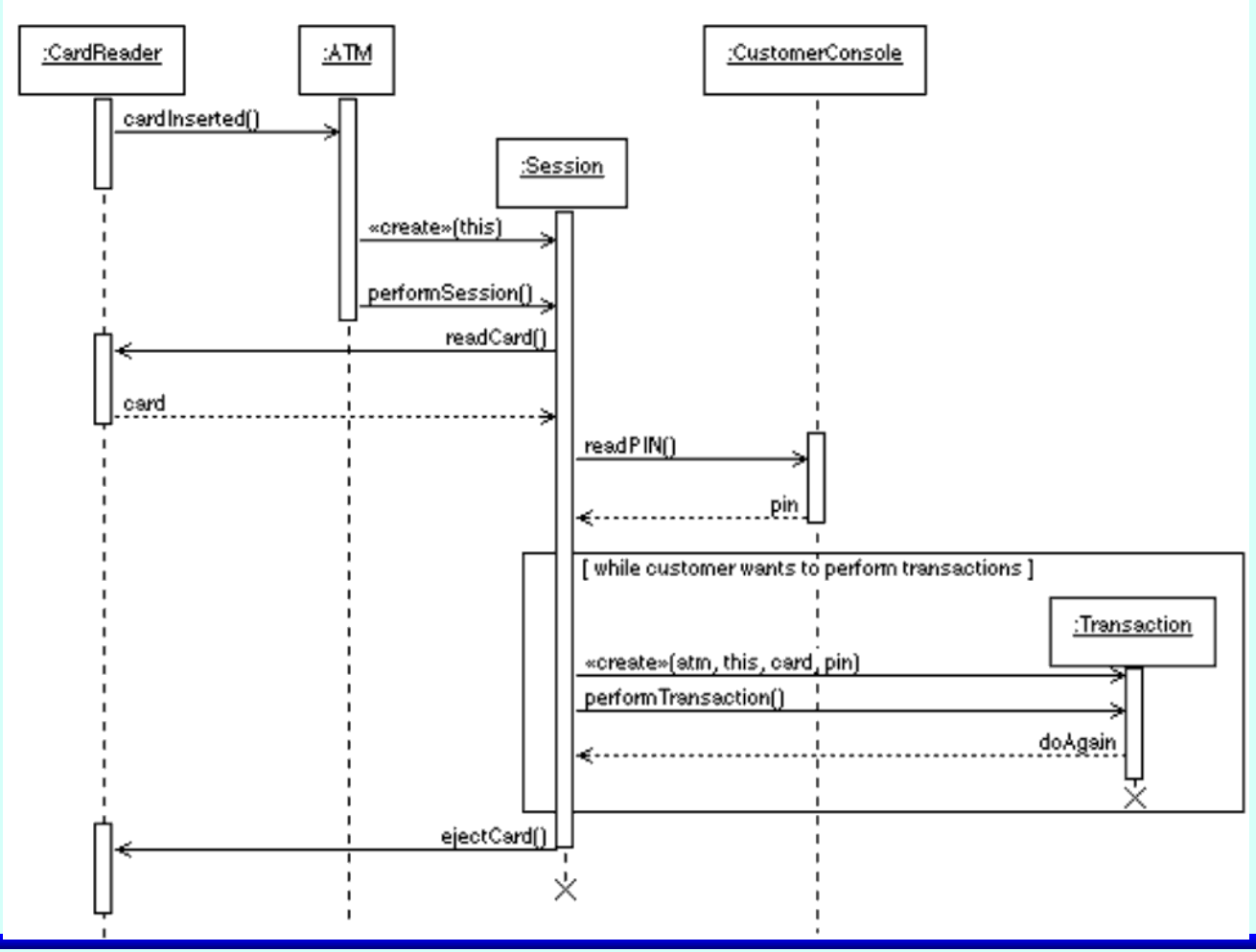
Fragments examples



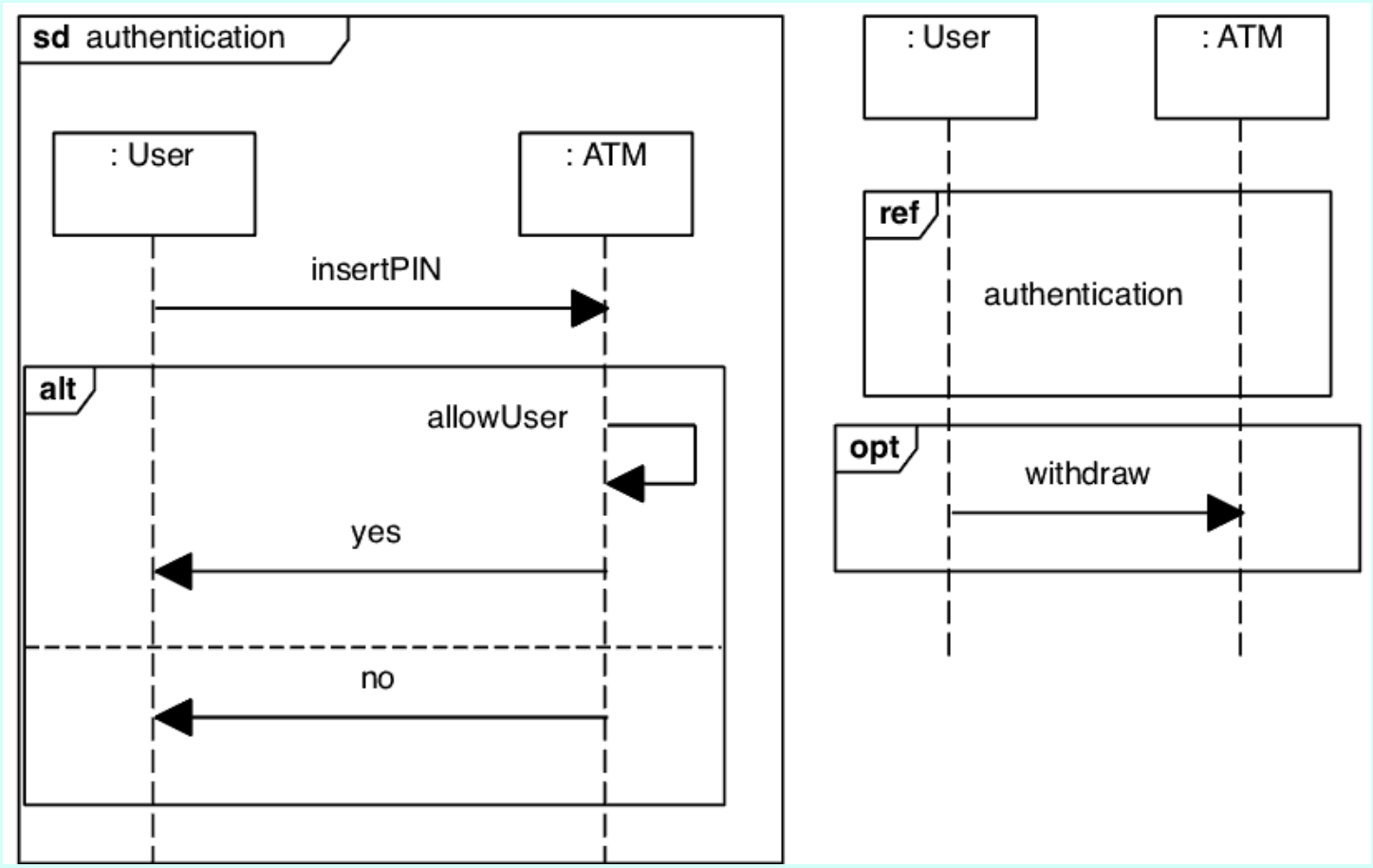
Fragments examples



Fragments examples (loop)



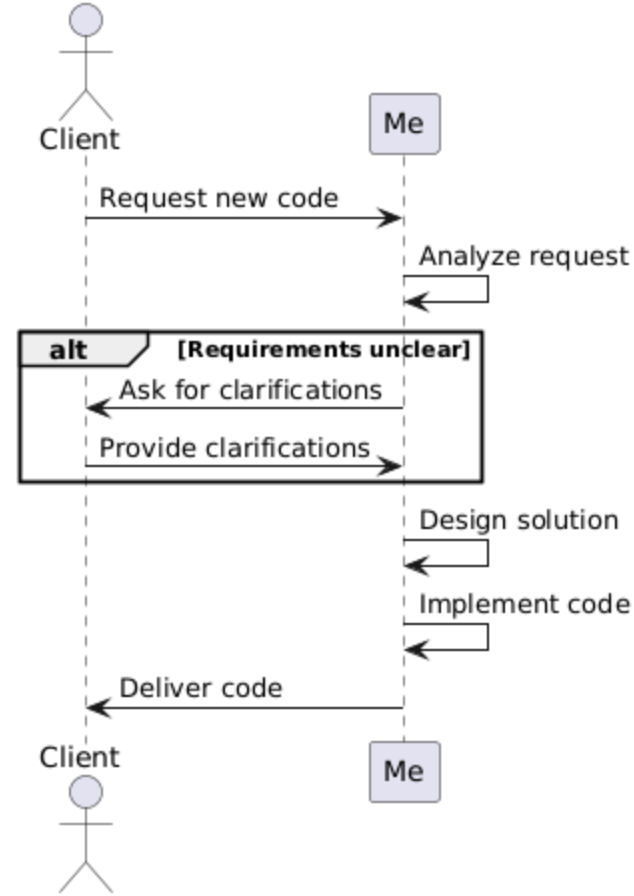
Fragments examples (break)



Exercise 2

- Draw the sequence diagram to model how you receive the request to develop a new piece of code

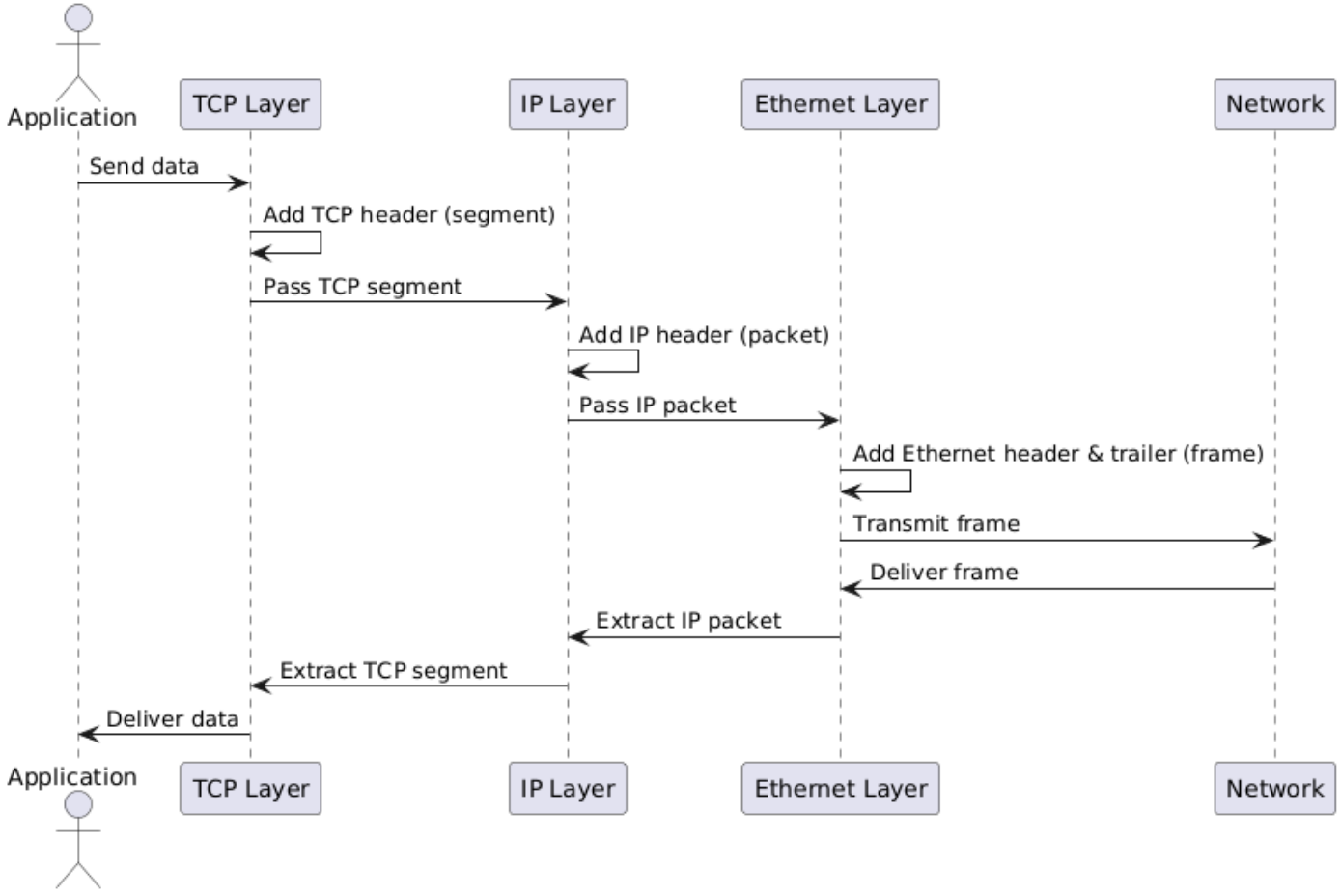
Exercise 2 -solution



Exercise 3

- Use the sequence diagram to model how a TCP packet is sent over an Ethernet connection

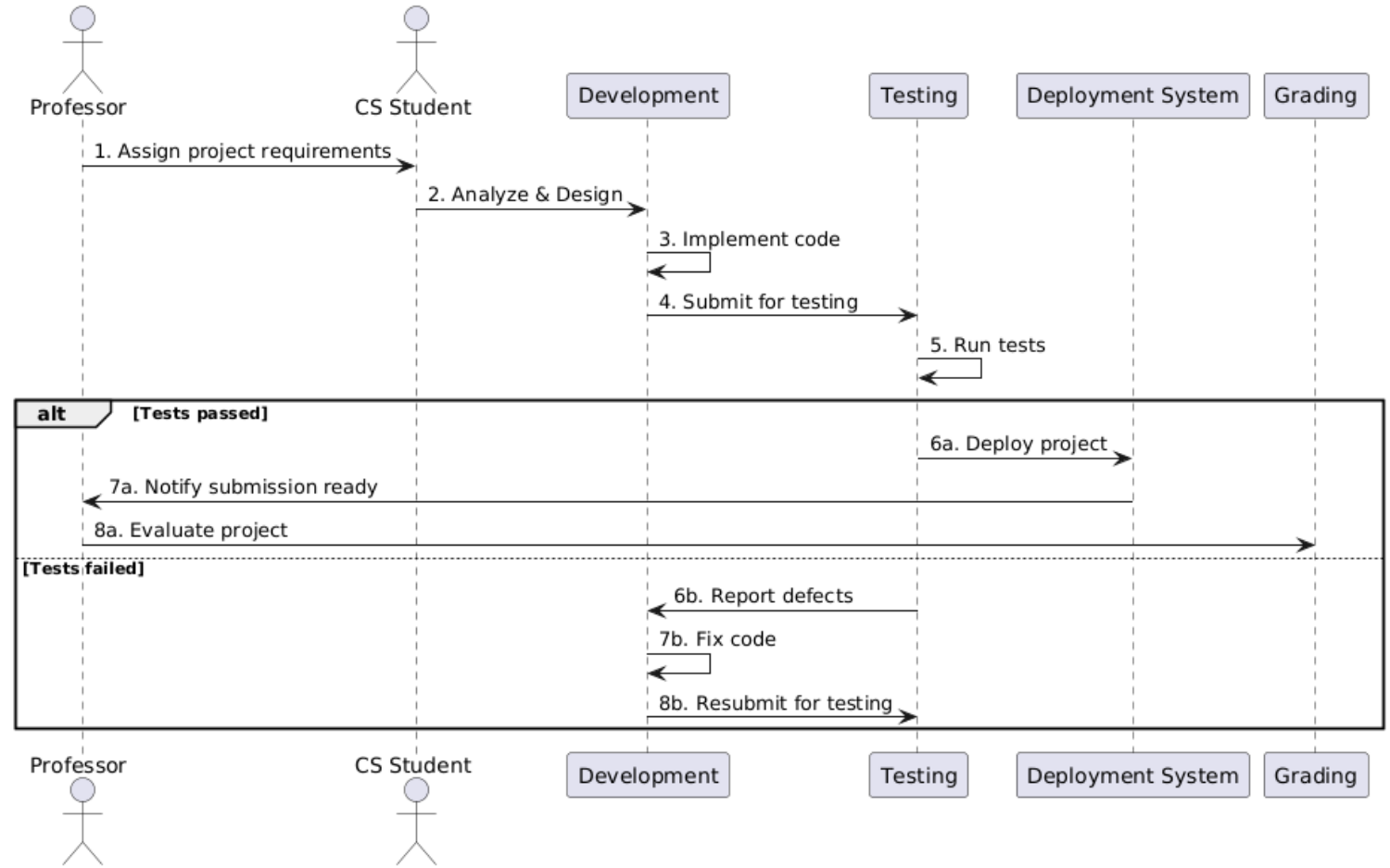
Exercise 3 - solution



Exercise 4

- Use the sequence diagram to model how you receive the request to develop a new piece of code -what is the difference with before?

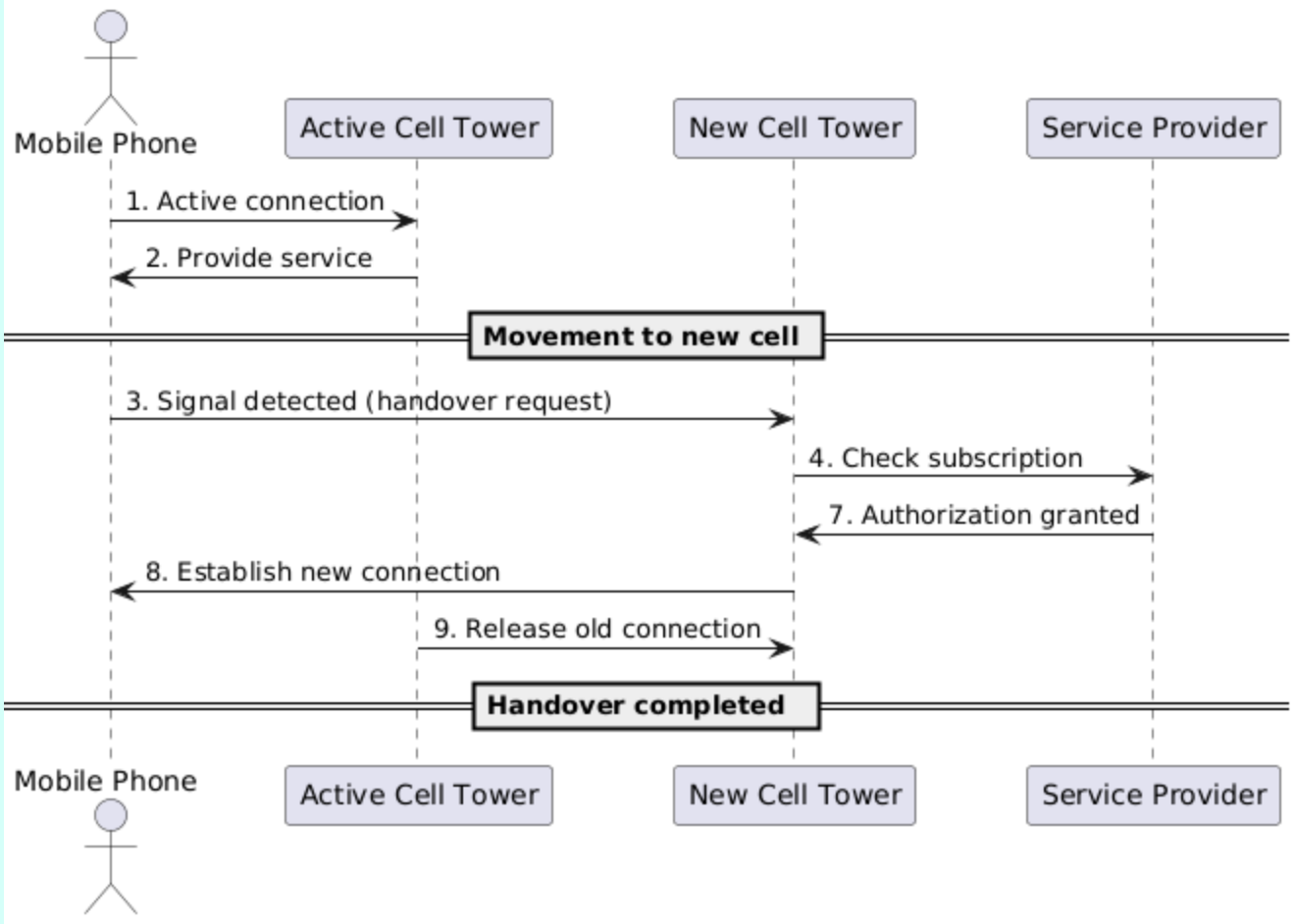
Exercise 4


Professor
CS Student
Development
Testing
Deployment System
Grading
1. Assign project requirements
2. Analyze & Design
3. Implement code
4. Submit for testing
5. Run tests
alt
[Tests passed]
6a. Deploy project
7a. Notify submission ready
8a. Evaluate project
[Tests failed]
6b. Report defects
7b. Fix code
8b. Resubmit for testing
Professor
CS Student
Development
Testing
Deployment System
Grading

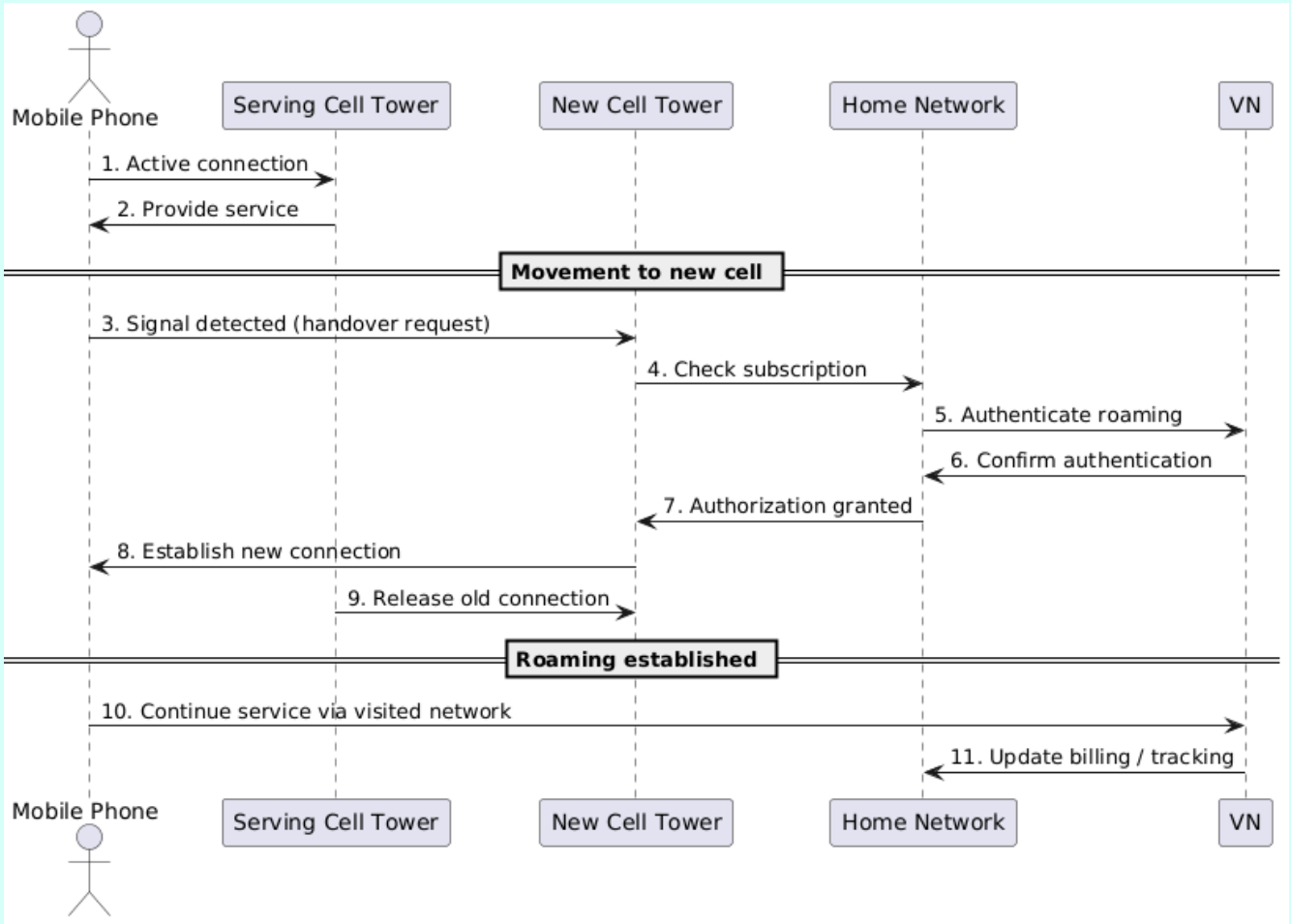
Exercise 5

Use the sequence diagram to represent how a cellular phone moves from cell to cell and handle roaming

Exercise 5 – cell to cell



Exercise 5 – roaming

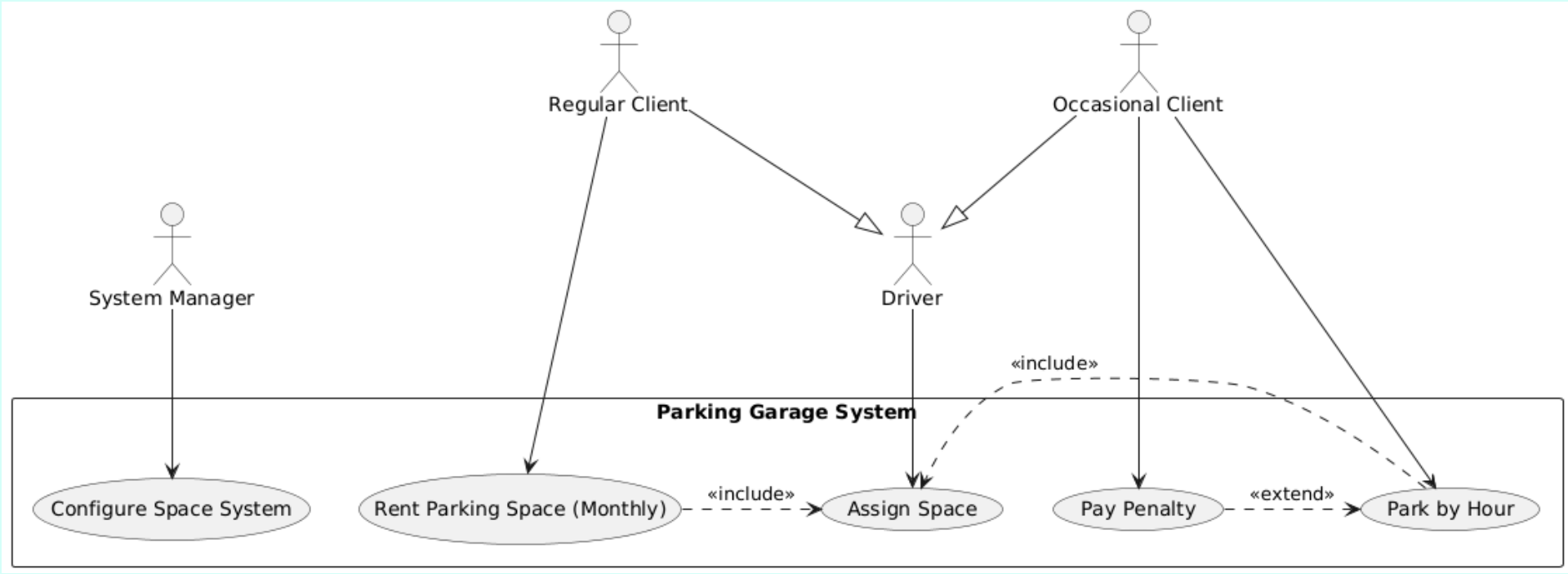


A Comprehensive Exercise: The Garage

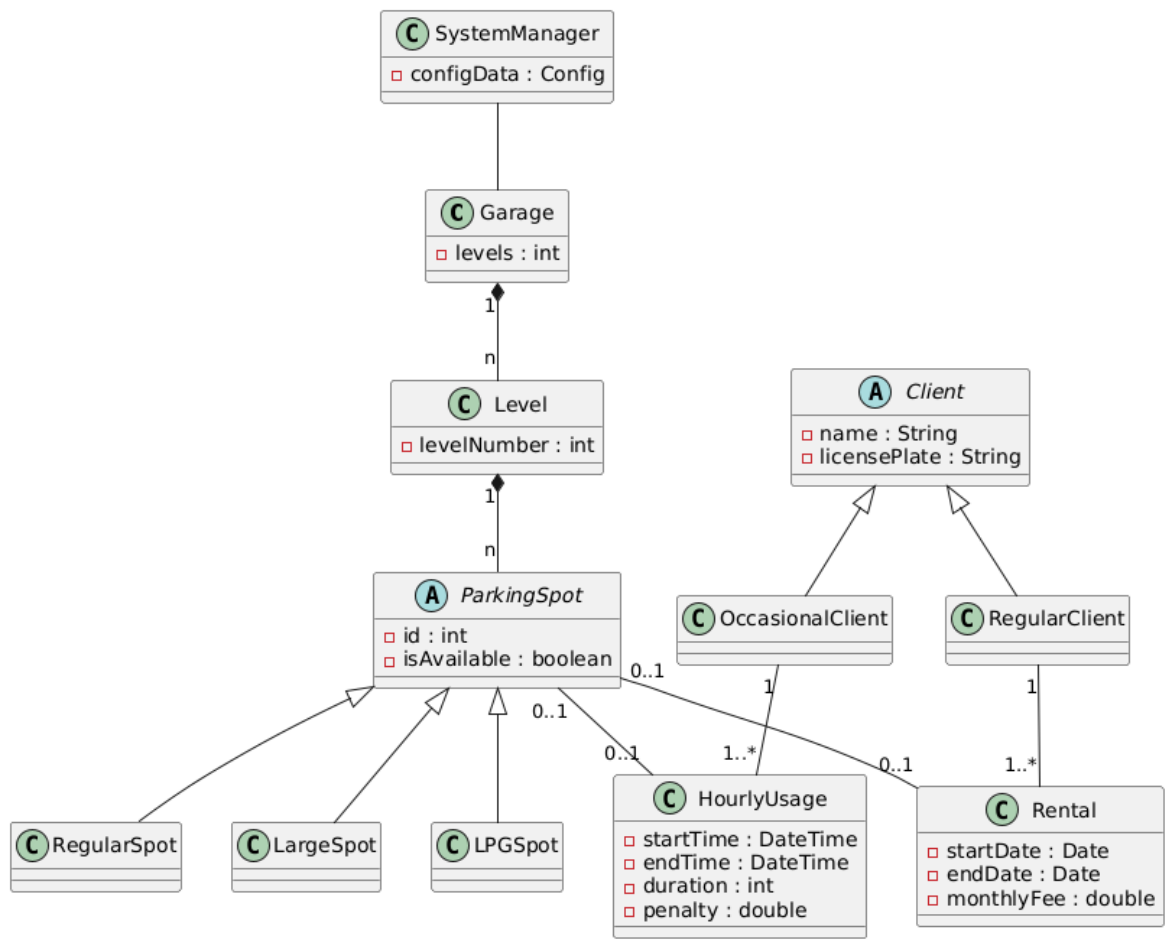
Problem

- A garage is made up of several levels. Each level has a number of available parking spaces. The spaces are of different types: regular cars, large vehicles (vans, …), luxury cars. LPG-powered cars can only park on the first floor.
- It is possible to rent a parking space, if available, on a monthly basis. No more than 50% of the spaces in each category can be rented. Spaces not rented on a monthly basis are used for hourly parking, up to a maximum of eight hours. If the eight hours are exceeded, a penalty is applied when the car is retrieved.
- The users of the system are both the drivers and the system manager, who provides configuration information (for example, the number of spaces in each category).
- **Provide: Use case diagram, Class diagram, Sequence diagram, State diagram**

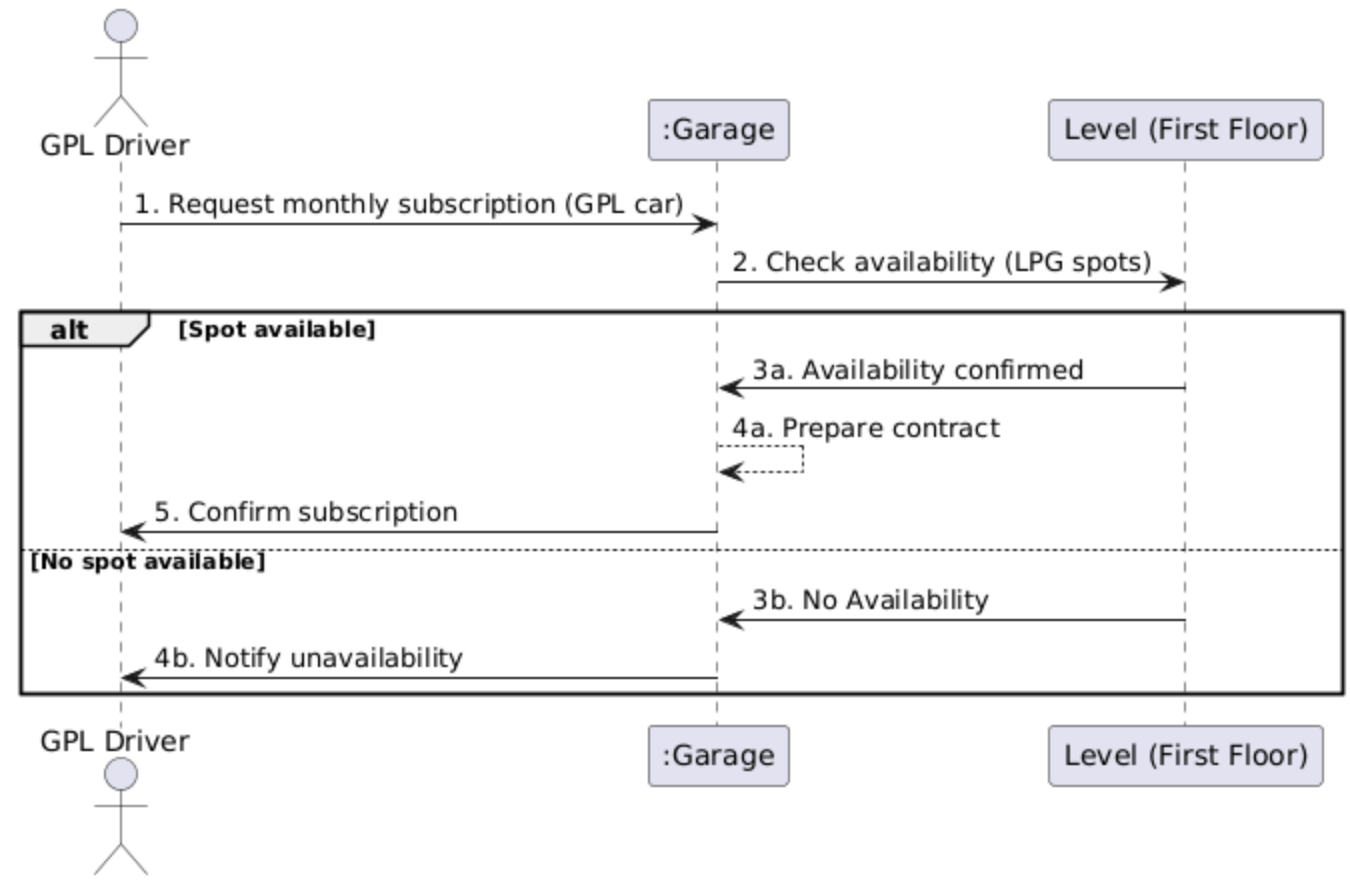
Use case


Regular Client
Occasional Client
System Manager
Driver
«include»
Parking Garage System
Configure Space System
Rent Parking Space (Monthly)
«include»
Assign Space
Pay Penalty
«extend»
Park by Hour

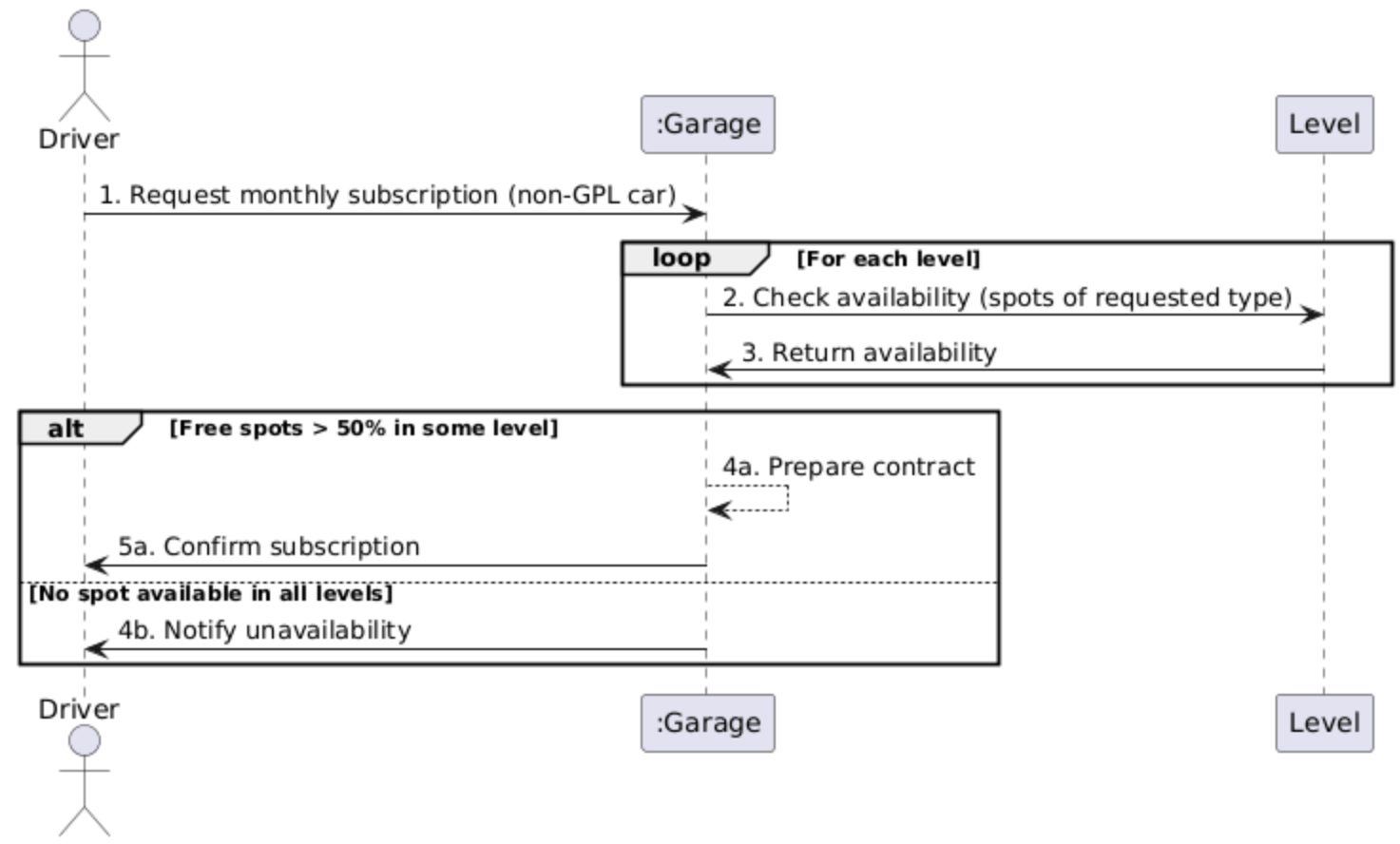
Classes


SystemManager
configData : Config
Garage
levels : int
1
n
Level
levelNumber : int
1
n
ParkingSpot
id : int
isAvailable : boolean
Client
name : String
licensePlate : String
OccasionalClient
RegularClient
0..1
1
1
0..1
0..1
1..*
0..1
1..*
RegularSpot
LargeSpot
LPGSpot
HourlyUsage
startTime : DateTime
endTime : DateTime
duration : int
penalty : double
Rental
startDate : Date
endDate : Date
monthlyFee : double

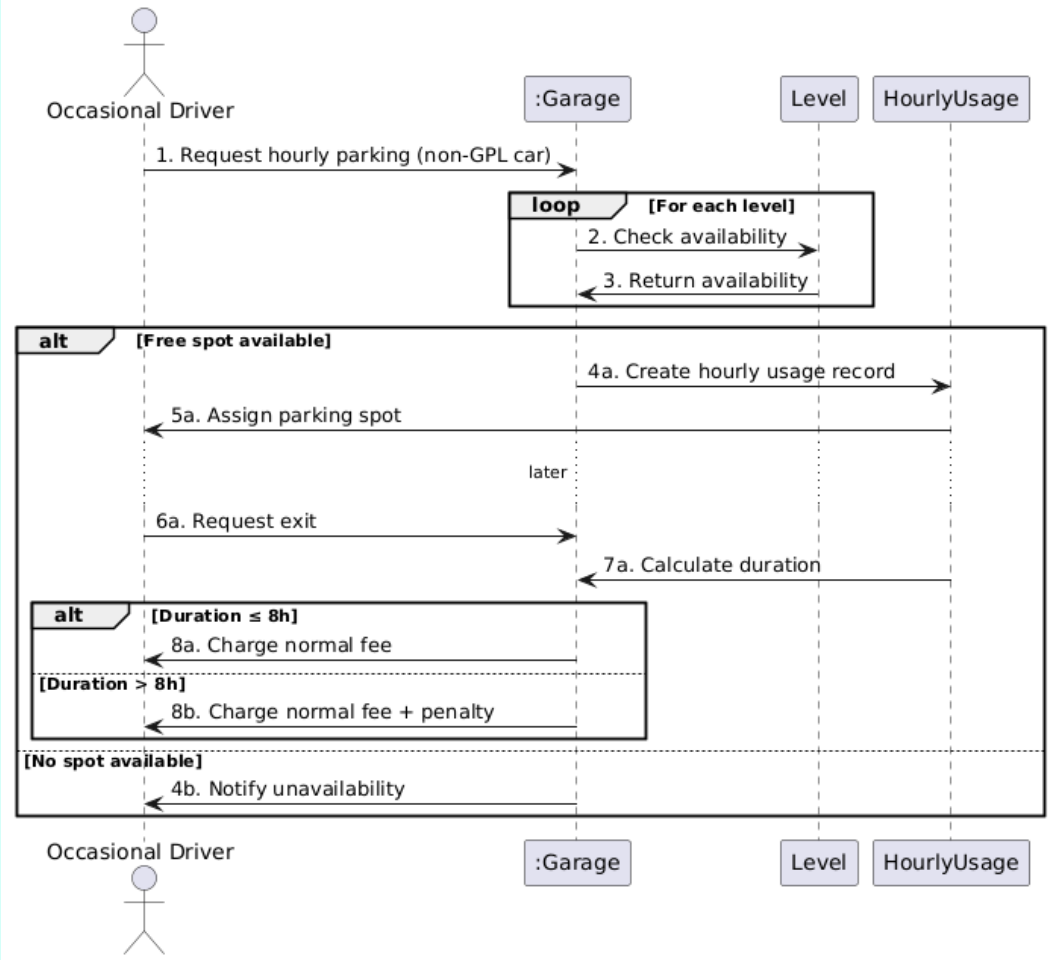
Sequence , GPL sub



Sequence , GPL sub



Sequence , occasional



Components and deployments

Component-exercise

A software company is building an online bookstore system.

The system has three main components:

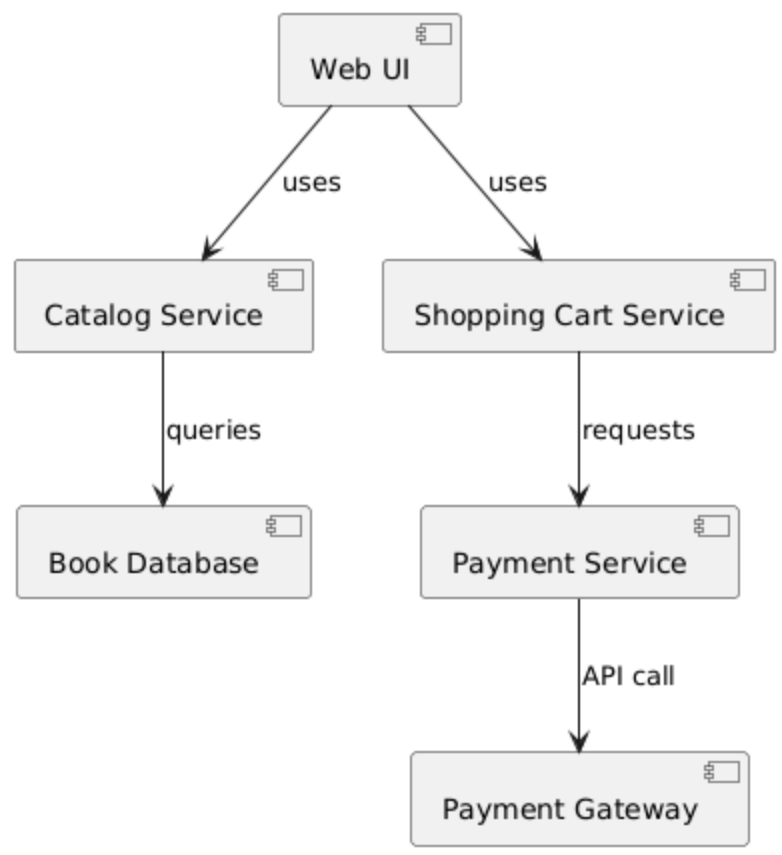
- Catalog Service: manages the list of books, their categories, and availability.
- Shopping Cart Service: allows customers to add/remove books, and stores cart state.
- Payment Service: processes payments through external payment gateways.

A Web UI component depends on the Catalog and Shopping Cart services.

The Shopping Cart Service depends on the Payment Service.

The Catalog Service uses a Book Database component to store and retrieve data.

Component-exercise solution



Deployment-exercise

Unibo wants to deploy a new online exam platform.

The system consists of a Web Application, a Database Server, and an Authentication Service.

The Web Application runs on an Application Server that communicates with both the Database Server and the Authentication Service.

Students and professors access the system from PCs and mobile devices through the Internet.

The Database Server is deployed on a separate physical machine, while the Authentication Service is hosted externally (e.g., Google OAuth).

The system must show how software artifacts (e.g., ExamApp.war, ExamDB, AuthAPI) are deployed on hardware nodes.

Deployment-exercise solution